

从零理解 simpleKafka

一门面向源码的中文渐进式课程

simpleKafka 教学项目

课程版本：Java 21 · 31 个可运行步骤

写在开始之前

本书用一章课程地图和八个实现章节，展开 31 步。读者会亲自完成记录格式、日志、网络请求、消费位点、消费组、受控副本切换，以及跨 producer、broker、consumer 的一致性实验；不是通过连接真实 Kafka 来“假装实现”，也不是把生产级 Kafka 缩成几个空洞名词。每一步都对应一个明确的 Java 方法、行为测试、失败边界和可重复命令。请从第一步开始按序推进；本书正文、课程清单与测试类共用相同编号。

课程代码刻意分成 `provided/`、`exercises/`、`reference/` 三个源根。学生入口编译前两者，参考入口编译第一和第三者；两套实现具有相同全限定类名，绝不可把两个答案源根放进同一次编译。参考实现用于在尝试后对照契约，并非学生测试 classpath 的后门。详细的开发环境、测试读取方法和晋级规则见本书第一章。

目录

第一章 课程地图、工具与学习方式	1
1.1 课程交付的边界	1
1.2 准备 Java 21 与课程命令	1
1.3 读懂二进制与文件 I/O	2
1.4 测试失败的阅读法	2
1.5 系统不变量先行	3
第二章 记录与单段持久化	4
2.1 第 1 步：记录编解码	4
2.2 第 2 步：日志追加	5
2.3 第 3 步：按位点读取	6
2.4 第 4 步：崩溃恢复	8
第三章 分段、索引、保留与截断	10
3.1 第 5 步：稀疏索引	10
3.2 第 6 步：分段日志	11
3.3 第 7 步：日志保留	13
3.4 第 8 步：尾部截断	14
第四章 分区与真实网络 broker	16
4.1 第 9 步：分区路由	16
4.2 第 10 步：主题元数据	17
4.3 第 11 步：TCP 帧协议	19
4.4 第 12 步：单机 broker	20
第五章 生产消费、位点与重复投递	22
5.1 第 13 步：批量生产者	22
5.2 第 14 步：拉取消费者	23

5.3	第 15 步：消费位点持久化	24
5.4	第 16 步：至少一次处理	26
第六章	消费组、重平衡与提交围栏	28
6.1	第 17 步：分区分配	28
6.2	第 18 步：成员与重平衡	29
6.3	第 19 步：心跳与过期	30
6.4	第 20 步：消费组围栏	32
第七章	拉取副本、ISR、高水位与确认	34
7.1	第 21 步：拉取复制	34
7.2	第 22 步：ISR 与高水位	36
7.3	第 23 步：生产确认	37
7.4	第 24 步：并发副本 broker	38
第八章	epoch 围栏、干净选举、分叉修复与重新入组	40
8.1	第 25 步：干净选举与 epoch	40
8.2	第 26 步：分叉日志修复	41
8.3	第 27 步：元数据路由刷新	43
8.4	第 28 步：副本重新入 ISR	44
第九章	生产者、broker 与消费者的一致性	46
9.1	第 29 步：三层一致性故障实验	46
9.2	第 30 步：幂等生产者	48
9.3	第 31 步：消费副作用幂等	51
附录 A	格式、错误码与接口索引	54
A.1	磁盘与网络格式	54
A.1.1	记录与索引	54
A.1.2	TCP frame 与请求体	55
A.1.3	消息 body 字段顺序	55
A.1.4	标识符与值类型	57
A.2	幂等追加与持久化副作用	57
A.3	固定错误码	57
A.4	三十一与编辑入口	58
A.5	课程简化与 Kafka 的差异	60

目录	iv
A.6 官方资料与延伸阅读	60
A.7 排版与构建说明	61
术语索引	62

第一章 课程地图、工具与学习方式

1.1 课程交付的边界

本项目把消息系统的主路径做成一个小而可运行的系统：磁盘上的分区日志、TCP 请求、生产与消费、消费位点、单 topic 消费组，以及多个本机 broker 之间的拉取复制和受控切换。学习对象不是 Kafka 官方客户端的实现，也不是 Kafka wire protocol。提供的二进制协议只服务本课程，不能把官方 `KafkaProducer` 或 `KafkaConsumer` 指向本程序。

三个 Java source root 是课程设计的一部分。`provided/` 放数据类型、I/O、传输、序列化样板、生命周期和测试夹具；它让注意力集中在算法而非重复搭框架。`exercises/` 是学生唯一需要逐步补全的工作区，所有步骤的类型和公开签名预先可见，尚未到达的方法会抛出带步骤号的 `ExerciseNotImplementedException`。`reference/` 是隔离的完整答案，供对照和课程验证；它与练习代码类名相同，因此学生和参考源根绝不能同次编译。测试在两种模式下复用同一套黑盒契约；学生运行时 `classpath` 不含参考 `classes`。

项目并非生产级 Kafka：没有 Kafka 线协议兼容、事务、压缩、compaction、完整控制器共识或 controller HA。第 8 章教学的是有限幂等生产者协议与本地持久化效果账本，它们不是 Kafka 生产协议，也不构成端到端事务。其 authority 是单 JVM 内的可信教学控制面；三台 broker 虽各有 TCP server、线程和磁盘目录，副本仍共享这个 authority。它可展示有控制的故障切换，不可用来声称任意网络分区下具备共识或生产级可用性。具体差异在附录和相关步骤反复标出。

1.2 准备 Java 21 与课程命令

项目使用 Gradle Wrapper `./gradlew`；Gradle 9.8.1 可运行于 Java 17+，Java 编译、测试和 demo 任务需要 Java 21+ JDK，并以 `--release21` 编译。不需要 Maven、Docker 或已启动的 Kafka。脚本使用 Bash 3.2+、`find`、`awk`、`curl` 与 `sha256sum` 或 `shasum`；准备 Tectonic 需要 `tar` 和 `install`，macOS 解压 Pandoc 还需要 `unzip`。EPUB 图形需要 Poppler 的 `pdftoppm`；`./gradlew :setupBook` 会检查它，但不安装系统软件。Linux/WSL 使用 `sudo apt-get install poppler-utils`，macOS 使用 `brew install poppler` 安装 Poppler。先检查 `java -version`、`javac -version` 与 `./gradlew --version`。可通过 `JAVA_HOME` 或 `PATH` 使用任意符合要求的 JDK；SDKMAN 可选，项目不会静默修改 SDKMAN、系统软件或 shell 配置。

`./gradlew :setup` 通过 Gradle 解析固定版本 JUnit 1.11.4 并验证 checksum，不再把 JUnit 放入 `.tools/`。`./gradlew :setupBook` 准备经过 checksum 校验的 Tectonic 0.17.0 与 Pandoc 3.12.1 到 `.tools/`，并检查 Poppler；不安装系统软件。`./gradlew :doctor` 检查 Gradle、Java、

Gradle 管理的 JUnit 与可选教材工具。./gradlew :compile 编译学生骨架与测试；编译成功不表示练习已经实现。课程清单与教材默认英文；设置 COURSE_LANG=zh 后运行 ./gradlew :list 可显示中文步骤标题。./gradlew :book 编译英文 PDF/EPUB 到 build/book/en/simpleKafka.pdf 与 build/book/en/simpleKafka.epub；设置 COURSE_LANG=zh 后运行同一命令，生成 build/book/zh/simpleKafka-zh.pdf 与 build/book/zh/simpleKafka-zh.epub。每次还会更新所选语言的 docs/book/dist/ 分发文件；读者可直接打开已分发成品，无需 Java 或编译工具。book 任务需要启动 Wrapper JVM，但不编译 Java 或解析 JUnit。首次构建需联网获取 TeX bundle；两种语言都在线成功后，设置 BOOK_OFFLINE=1 并用 ./gradlew :book --offline 离线构建。Gradle --offline 控制 Gradle 依赖；BOOK_OFFLINE=1 为两次 Tectonic 调用启用 TeX bundle only-cached 模式。

1.3 读懂二进制与文件 I/O

ByteBuffer 是一块带 position/limit 的字节视图。读取前通常要把写入完成的 buffer flip()；写出后是否清理取决于是否继续复用。协议和磁盘布局规定大端序，因此编码端与解码端都必须遵守相同字节次序。逻辑 offset 表示第几条记录；position 表示文件中的第几个字节，两者数值碰巧相同也毫无含义。

FileChannel 能在指定文件偏移读写，也能把已写数据请求强制同步。课程提供了处理部分读写的 ChannelIO.readFully/writeFully；一次 channel.read 并不保证填满 buffer，一次 write 也不保证写完。步骤 2 选择每批追加后 force(false)，意味着要求文件数据持久化的一种简化，不是对掉电、电源缓存、介质损坏或文件系统语义的数学保证。CRC32C 检测记录字节意外改变；它不修复错误，也不能证明数据来源可信。

1.4 测试失败的阅读法

先读失败的 Step 测试名和断言，再回到本章契约。测试报告中的第一处异常通常比最后一段堆栈更有用：未实现方法给出步骤和方法名；课程错误应保留 ErrorCode；意外 IOException 可能被包装为 STORAGE_ERROR 并保留 cause。不要把错误映射成空列表或成功默认值来让测试变绿。一个单步定位命令只运行该步，不代表累计依赖都满足；累计命令会再次评估之前每步。课程 runner 按步骤分组输出测试名称、各步和总计数；失败时显示断言、精确方法定位与重跑命令。报告始终为英文；legacy JUnit XML 保存到 build/student/reports/ 或 build/reference/reports/。自定义 JavaExec 课程任务输出纯文本。标准 Gradle :exercises:test 与 :reference:test 支持 IDE 调试及精确 --tests 选择；其 XML/HTML 报告位于 build/<mode>/gradle-test-results/ 与 build/<mode>/gradle-test-reports/。runner 保留 NO_COLOR 与 TERM=dumb 颜色门禁。

命令	意义
<code>./gradlew :list</code>	显示 31 步、编辑方法与前置关系。
<code>./gradlew :stepTest -Pstep=12</code>	只定位第 12 步；失败时辅助找当前问题。
<code>./gradlew :test -Pstep=12</code>	累计运行 Step01Test 至 Step12Test。
<code>./gradlew :referenceTest -Pstep=31</code>	对隔离参考实现运行完整 31 步契约。
<code>./gradlew :referenceDemo</code>	运行完整参考 TCP/failover 场景。
<code>./gradlew :referenceConsistencyDemo</code>	运行第 8 章真实 TCP 与磁盘一致性场景。

步骤编号接受 1-31 的十进制数字，可有前导零（例如 `-Pstep=1` 或 `-Pstep=01`）。章节边界是第 4、8、12、16、20、24、28、31 步：只有该边界累计测试全绿，才算章节完成。`./gradlew :test -Pstep=8` 通过意味着第 1-8 步全部通过；`./gradlew :stepTest -Pstep=8` 仅定位第 8 步，不授予晋级。全课程测试不应把参考源放进学生 classpath，参考模式也不能代替自己的练习进度。

1.5 系统不变量先行

阅读后续章节时牢记以下四条边界：一，日志保留范围内 offset 连续，LEO 是下一条写入位点，fetch 使用左闭右开区间；二，普通消费者可见范围不能越过 HW，group committed offset 是另一个状态量；三，锁按 authority partition lock 到本地 log lock 的顺序获取，网络 I/O 不在该锁内等待；四，错误必须有意义，尤其 epoch 或 generation 失效时不以静默重试伪造 exactly-once。每章都会说明它新引入了什么不变量以及由谁验证。这些不变量将在具体步骤落地：磁盘格式见第 1 步（记录编解码），日志尾部边界见第 4 步（崩溃恢复），消费位置见第 14 步（拉取消费者），复制可见性见第 22 步（ISR 与高水位），角色围栏见第 25 步（干净选举与 epoch）。

第二章 记录与单段持久化

这一章回答一个基础但关键的问题：消息写进文件后，如何知道它完整、顺序正确，并能在进程重启后恢复？一个 Kafka partition 在概念上是只追加的日志；本课程先把每条记录直接编码到一个单段文件，然后再加入分段、索引和保留。这里的 offset 是逻辑位置，不能拿文件字节位置替代。

length	CRC32C	offset	timestamp	key/value len	key value
4 B	4 B	8 B	8 B	8 B	变长

图 2.1: 一条磁盘记录。length 不包含最前面的 4 字节自身。

2.1 第 1 步：记录编解码

目标与前置知识

本步目标：定义稳定的磁盘边界，使写入和读取方对字节有同一解释。

开始前应能解释：理解有符号长度、固定宽度整数、UTF-8 字节与 CRC32C；不要求先会文件扫描。

背景与推理

格式为大端 `length:int32|crc32c:int32|offset:int64|timestamp:int64|keyLength:int32|valueLength:int32|key|value`。length 从 CRC 字段开始计，合法范围 28 到 1,048,576；keyLength=-1 表示 null，0 表示空 key。value 必须存在但允许长度 0。CRC32C 覆盖 offset 字节起至 value 末尾；length 前缀和 CRC 字段本身不参与校验。offset/timestamp 均非负。先验证长度再按声明字段取字节，才能区分恶意/损坏长度与真正不足的 buffer。解码恰消费一条记录，即便后面还有下一条记录也不能把后缀吞掉。

起点与编辑位置。

已给：LogRecord、RecordData、CourseException、ErrorCode、大端字段与 CRC32C 工具类型已经提供。数组所有权交给数据类型；不要为每次字段访问重复拷贝。

请修改：`exercises/src/main/java/io/simplekafka/storage/RecordCodec.java`：
`encode(LogRecord)`、`decode(ByteBuffer)`。

行为契约与不变量。编码应精确分配所需记录空间，记录的 offset/timestamp 不可为负，value 不可 null。解码输入不足是 INVALID_REQUEST；格式错误或 CRC 不匹配是 CORRUPT_RECORD。成功后 buffer position 正好位于该条记录之后，后续 bytes 原样留待下一次解码。Null key 与空 key 不可混淆。

分级提示（不是答案）

- 一级 把格式表按“定长头部”和“变长尾部”拆开，先列每个字段的字节数。
- 二级 CRC 检验时明确它保护的起止边界；length 的定义与总分配字节数不是同一个量。
- 三级 编写往返和单字节破坏测试，再分别观察短 buffer、非法长度与校验失败应该属于哪类协议错误。

可观测例子与测试点。编码并解码一条 offset=7、key=null、timestamp=9 的记录，value 为“值”的 UTF-8 字节。解码后这些字段都应保持不变。value 任一位翻转都会报 CORRUPT_RECORD；输入不足一条完整记录时报 INVALID_REQUEST。这些对应 Step01Test，也验证解码未吞掉紧随其后的记录。

常见误区与真实 Kafka 的差异

把长度前缀计入 length、把 CRC 覆盖到自身、把 null key 解释为空数组，都会破坏互操作。CRC 是损坏探测而非安全签名。真实 Kafka 通常按 record batch 组织并有更丰富的压缩与批量格式；课程故意用单记录格式让字段边界可见，不宣称 Kafka 磁盘格式相同。

命令与晋级。 ./gradlew :stepTest -Pstep=1 用于定位本步； ./gradlew :test -Pstep=1 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么 record 的逻辑 offset 不能从文件 byte position 推算？
2. CRC 校验成功能证明哪些事情，不能证明哪些事情？

2.2 第 2 步：日志追加

目标与前置知识

本步目标：实现单段日志的连续 offset 分配与可观察持久化。

开始前应能解释：掌握第一步的编码，以及锁、全写循环和文件位置写入的区别。

背景与推理

追加把数据从内存转为有序的日志事实。partition 的本地锁串行化同一分区的追加，使竞争批次不会获得交错或重叠的 offset。一次普通 FileChannel.write 可能只写部分 buffer，因此必须使用提供的 ChannelIO.writeFully。批次写完后调用 force(false)，再公布追加结果；

`AppendResult(firstOffset,nextOffset)` 使用左闭右开区间，例如 `[0,3)` 表示三条记录。写入过程遇到 I/O 错误时课程不保证整批原子回滚，重启恢复负责识别物理尾部残片。

起点与编辑位置。

已给：SegmentLog 的 path、FileChannel、锁、baseOffset、nextOffset 与 close() 已给定；记录编码和部分写 I/O 已可复用。

请修改：exercises/src/main/java/io/simplekafka/storage/SegmentLog.java: append(List<RecordData>)。

行为契约与不变量。空 batch 报 INVALID_REQUEST。非空 batch 在锁内以当前 LEO 为首 offset 分配连续编号；整批先做参数及单记录大小预检，不能让已知非法输入写入前缀后再失败。写出全部记录并 force 成功后返回首 offset 与下一 offset。每批记录的 timestamp/数据合法性由既定类型和 codec 契约约束。

分级提示（不是答案）

- 一级 用一个 batch 的首尾 offset 表达结果，不把“记录数”误作下一批首 offset。
- 二级 想清楚哪类错误在取得锁以前就能判断，哪类状态必须在锁内读取。
- 三级 让并发追加只竞争同一个日志实例，并观察批次级次序与文件内容；别只断言最终 LEO。

可观测例子与测试点。空段依次追加三条、再追加两条，预期结果分别为 `[0,3)` 与 `[3,5)`，从文件解码得到 offset 0、1、2、3、4。两个并发两条记录 batch 的结果区间必须不相交，且总记录数为四。对应 Step02Test 的实际文件与并发检查。

常见误区与真实 Kafka 的差异

先写再发现某条记录超长，会留下不应存在的部分 batch。只更新 nextOffset 而不 force/检查写入同样错误。真实 Kafka 采用批处理、后台 I/O 与更复杂的持久化策略；本课程每次追加显式 force 以便观察边界，这不是 Kafka 默认每条消息同步落盘的性能模型。

命令与晋级。 ./gradlew :stepTest -Pstep=2 用于定位本步；./gradlew :test -Pstep=2 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为何验证整个 batch 应发生在任何磁盘写入之前？
2. force 成功代表什么，又不代表什么？

2.3 第 3 步：按位点读取

目标与前置知识

本步目标：从指定 offset 读取完整记录，并同时尊重记录数和字节预算。

开始前应能解释：区分 offset、段内 byte position、记录完整性以及游标状态。

背景与推理

消费者请求的是逻辑位点，文件读取需要物理 byte position。第一版公开读取从段首开始；下一章索引会把一个逻辑位点附近的物理位置提示给日志。提示只能缩短搜索，不能改变 offset 语义。每条记录是不可拆的返回单位：若下一条完整记录放不进预算，就不返回半条。预算统计包含 length 前缀。显式 position 的 FileChannel 读取避免 read 与 append 共享可变 channel cursor。

起点与编辑位置。

已给：段生命周期和记录解码已完成；公开 `read(offset,maxRecords,maxBytes)` 会把默认 hint 传给内部方法。

请修改：`exercises/src/main/java/io/simplekafka/storage/SegmentLog.java: readFrom(longoffset,longbytePositionHint,intmaxRecords,intmaxBytes)`。

行为契约与不变量。参数 offset 可从 baseOffset 到 LEO (含 LEO 的空读)；段内超出有效范围报 `OFFSET_OUT_OF_RANGE`。maxRecords/maxBytes 必须为正。返回记录按 offset 连续且不超过两个预算；第一条自身超 byte budget 时结果为空。有效 hint 必须位于记录边界且不能越过目标 offset，否则报 `INVALID_REQUEST`。读取不得移动追加位置或改变 LEO。

分级提示 (不是答案)

- 一级 把“找到候选字节位置”与“输出应从哪个 offset 开始”视为两个独立问题。
- 二级 为 byte budget 计算完整记录占用，并考虑 length 前缀；不按 payload 字节数近似。
- 三级 设计一个 hint 指向目标之前的边界、另一个越过目标的边界，分别验证可用和拒绝条件。

可观测例子与测试点。追加 offset 0-4 后，`read(2,2,4096)` 得 offset 2、3。再调用 `readFrom(2,hint,2,4096)`，把 hint 设为 offset 1 记录在文件中的实际起始字节位置，仍应返回 offset 2、3；hint 是物理位置，不是逻辑 offset 数字。`read(5,2,4096)` 返回空；从 offset 1 以 66 byte 预算读取恰好返回 1、2，以更小预算只返回 1。第一条完整记录超过 maxBytes 时返回空而不是部分 bytes。对应 `Step03Test`。

常见误区与真实 Kafka 的差异

offset 2 不是文件第 2 字节；将两者混为一谈会在索引出现后随机丢记录。忽略前缀预算会超过调用者内存预算。真实 Kafka fetch 通常以 batch 与网络字节上限协商，本实现以单记录为原子返回单位，消费跨段的总预算由下一章统一处理。

命令与晋级。`./gradlew :stepTest -Pstep=3` 用于定位本步；`./gradlew :test -Pstep=3` 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 稀疏索引为什么只能提供搜索起点，不能直接决定读取结果？
2. 第一条记录比 maxBytes 大时，返回部分记录会造成什么问题？

2.4 第 4 步：崩溃恢复

目标与前置知识

本步目标：从一个可能在写入中途关闭的日志恢复连续 LEO，并仅修复可证明是不完整尾部的情况。

开始前应能解释：理解记录边界、物理截断和错误类别。

背景与推理

崩溃可能发生在 length 前缀、固定头或 payload 写入期间。若 EOF 位于最后一条的 header/body 内，完整前缀之前的数据仍可构成日志，可截断到上一条边界；但一个完整记录的 CRC 坏、长度非法、offset 不连续，都不是“安全丢弃的尾部”。将完整但损坏的证据静默截掉，会把介质故障误报成成功恢复。恢复从 baseOffset 建立期望 offset，扫描到最后一个有效边界并返回下一个可写位点。

起点与编辑位置。

已给：SegmentLog.open(file,baseOffset) 会打开已有文件并调用恢复；步骤 1 codec 与步骤 2/3 文件布局已经建立。

请修改：exercises/src/main/java/io/simplekafka/storage/SegmentLog.java: recover()。

行为契约与不变量。成功时返回恢复后的 LEO，下一次 append 从它继续；只允许截断文件末尾的不完整记录。完整记录 CRC 错、长度不合法或 offset 断裂必须以 CORRUPT_RECORD 失败，不得跳过错误记录继续扫描。底层 I/O 异常使用 STORAGE_ERROR 并保留 cause。

分级提示（不是答案）

- 一级 先定义每一个扫描成功后的“共同状态”：当前有效字节边界与下一期望 offset。
- 二级 将 EOF 不足字节与已读满却内容不正确的记录分开分类。
- 三级 构造半条尾记录与完整 CRC 损坏记录的两类 reopen 测试，确认只有前者改变文件长度。

可观测例子与测试点。文件含完整 offset 0、1，再含 offset 2 的半条 body；重新 open 后 LEO=2，文件恰截到第二条结尾；追加一条得到 offset 2。若 offset 1 完整但 CRC 错，reopen 应失败而不把它当作残尾。对应 Step04Test。

常见误区与真实 Kafka 的差异

“扫描到报错就截断”会吞掉完整记录损坏和格式错误；“任何 EOF 都是无效文件”又无法处理正常崩溃尾部。force(false) 提供的是本课程写入意图，不是断电安全证明。真实 Kafka 还有 segment recovery、索引修复、日志校验与更复杂的清理策略；本步骤只恢复一段的边界。

命令与晋级。 ./gradlew :stepTest -Pstep=4 用于定位本步；./gradlew :test -Pstep=4 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 哪些字节证据足以判定尾部不完整，哪些必须当成损坏上报？
2. 为什么恢复应返回 LEO 而不是最后一条记录的 offset？

第三章 分段、索引、保留与截断

单文件日志随时间增长，会令启动扫描越来越慢，也无法单独删除旧数据。Kafka 风格的日志用多个 segment 组成一个分区：旧 segment 封闭，当前段承接追加；稀疏索引把逻辑 offset 映射到某个段内字节边界。索引是可重建的加速结构，不是记录真相。日志保留和截断是不同操作：前者删除旧的封闭前缀，后者把尾部回退到某个位点以修复分叉。

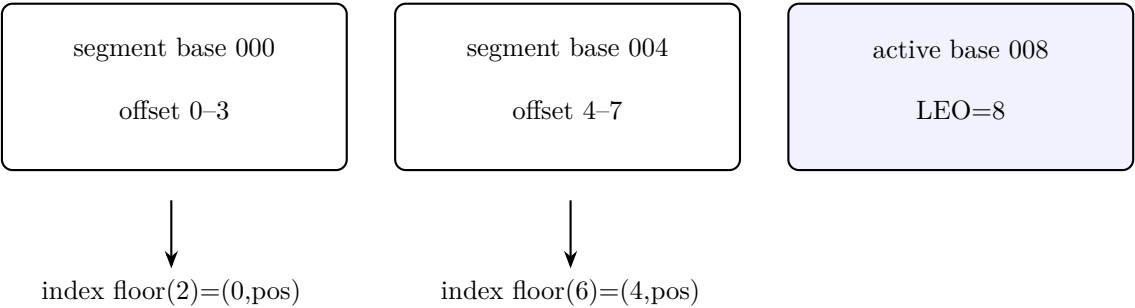


图 3.1: 分段日志的逻辑范围与段内稀疏索引。段文件名中的 base offset 是该段逻辑起点。

3.1 第 5 步：稀疏索引

目标与前置知识

本步目标：构建可恢复的 offset 到 byte-position 稀疏映射。

开始前应能解释：理解 floor 查找、记录边界与日志扫描恢复。

背景与推理

索引项格式是两个大端 int64: `offset, position`，每项 16 字节。每隔 `intervalRecords` 条记录写一个点，第一条记录总要有索引项。查找目标 offset 时，`floor` 返回不大于目标的最大的 offset，使读取可从近处合法边界开始并继续顺扫。不存在合适项时，`(baseOffset, 0)` 是安全起点。索引可能丢失、截短或含非法项；主日志仍是事实来源，恢复时应扫描日志重建索引。

起点与编辑位置。

已给：索引文件路径、段 `baseOffset`、`interval` 和生命周期已在 `SparseIndex` 提供；`SegmentLog` 可从记录重扫。

请修改：`exercises/src/main/java/io/simplekafka/storage/SparseIndex.java`：
`add(long, long)`、`floor(long)`、`rebuild(Path, long)`。

行为契约与不变量。添加项按 offset 严格递增且 position 非负；floor 返回最大 offset ≤ 目标，找不到时返回段起点的零位置。索引缺失、文件长度不是 16 的倍数或条目非法时从日志重建；日志本身损坏仍须失败。重建不能把错误索引伪装成可信日志数据。

分级提示（不是答案）

- 一级 先写清楚 floor 的数学定义：候选集合是哪些 offset，空集合的约定是什么。
- 二级 把索引读取失败分成“索引可丢弃”和“主日志不可恢复”两类。
- 三级 用多段记录构造间隔索引，删除索引文件后再通过真实读取观察恢复效果，而不只检查内存 map。

可观测例子与测试点。interval=4 时，对 offset 0、4、8 建项；floor(6) 返回 offset 4 对应的物理位置。删去 .index 后 reopen 并读取 offset 6，仍得到正确记录。非法长度索引文件也可重建；有损坏日志仍报错。对应 Step05Test。

常见误区与真实 Kafka 的差异

索引项若指向记录内部，读取会把任意 payload 当作 header；floor 误用 ceiling 会跳过目标前的边界。索引若成为唯一数据来源，缺索引就丢日志。Kafka 的索引组织与精度/字节偏移模型更复杂；课程每 intervalRecords 条记录索引一次，不是按固定字节间隔。

命令与晋级。./gradlew :stepTest -Pstep=5 用于定位本步；./gradlew :test -Pstep=5 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么找到 floor entry 后仍要扫描记录？
2. 损坏的 index 和损坏的 log 为何不能采用同一种恢复策略？

3.2 第 6 步：分段日志

目标与前置知识

本步目标：把单段日志组合成可滚动的分区日志，并让读取跨 segment 连续工作。

开始前应能解释：理解段边界、容量估算、索引与全局 LEO。

背景与推理

PartitionLog 按 baseOffset 排序持有多个 SegmentLog。活动段若容纳不了下一条合法记录，就封闭当前段并新建 base= 当前 LEO 的段；单条记录即使大于配置 segmentBytes，只要记录自身合法，也应独占新段而非永远无法追加。一次 batch 可以横跨段边界，offset 在整个 partition 范围连续。读取先定位目标段，然后用该段索引 floor 得到提示，顺扫后如有需要进入下一段，并

把 `maxRecords/maxBytes` 作为跨段总预算。创建新文件前需要预检整个 batch 的合法性，避免已知非法输入改变目录结构。版本标记在整批预检后、首次写盘或滚段前递增一次；无效输入不增加版本，写入尝试即使之后遇到 I/O 错误也已使旧 proof 失效。

起点与编辑位置。

已给：每个 segment 的 `create/open/recover`、稀疏索引与资源关闭都已提供；`PartitionLog` 骨架提供 `mutationVersion()` 与私有 `markMutation()`，追加实现必须在正确位置调用。

请修改：`exercises/src/main/java/io/simplekafka/storage/PartitionLog.java`：
`append(List<RecordData>)`、`read(long,int,int)`。

行为契约与不变量。 `append` 返回整个 batch 的 `[first,next)` 区间，跨段 offset 仍无间隙。容量不足时只在追加前滚段；非法 batch 不得先滚段。读操作按 offset 顺序跨段，累计总记录与字节限制；段直接追加本身不负责滚动。重开已存在目录后恢复 LEO，不重复分配旧 offset。目录发现只处理约定 `.log/.index` 文件，不删除陌生文件。每个有效追加批次对本打开的日志对象 `mutationVersion` 加一；这只是内存代数，不写入磁盘。

分级提示（不是答案）

- 一级 容量判断应该使用编码后的完整记录字节数，而不是只计算 value 长度。
- 二级 决定一次 batch 的每条记录与段边界如何配合，同时保持全局 offset 连续。
- 三级 将测试分成初次 `append`、滚动后 `append`、关闭重开后的 `append`，再逐条核对所有 offset。

可观测例子与测试点。 配置 256 字节段并追加十条记录，关闭重开再追加一条，读取范围应仍为 0 到 10 且没有重复 offset。测试也覆盖 batch 正好跨段、单条记录独占新段、非法超长记录前后目录/LEO 不变；对应 `Step06Test`。

常见误区与真实 Kafka 的差异

容量只看原始 payload 会漏算固定头、长度和 CRC；只让每个 batch 单独检查 offset 会造成跨段冲突。真实 Kafka segment 更常按字节规模滚动并结合索引文件和时间策略；课程也用容量滚动，但索引间隔是记录数。单段 `append` 不自动滚段是有意分层。

命令与晋级。 `./gradlew :stepTest -Pstep=6` 用于定位本步；`./gradlew :test -Pstep=6` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为何要在创建任何新段之前验证整批记录？
2. 跨段 fetch 的字节预算应如何避免在段切换时被重置？

3.3 第 7 步：日志保留

目标与前置知识

本步目标：安全删除已过期的封闭段，同时保留 active segment 与当前 LEO。

开始前应能解释：区分日志起点、消费进度和单条删除。

背景与推理

分区日志实际只有一段可保留范围 [logStartOffset, LEO)。课程用显式调用进行保留，不运行后台定时器。deleteBefore(offset) 只删除最后记录 offset 小于阈值的封闭段，不能按记录切割一段，也不能移除活动段；新起点必须落在真实段边界。消费者若请求已经删除的数据会得到 OFFSET_OUT_OF_RANGE，不静默把 position 改到新起点。消费组 offset 提交与物理删日志独立，推进消费位点不会自动触发保留。只有确实删除至少一个段时才递增一次本日志对象的 mutationVersion；阈值无效或没有可删段的调用不变更版本。

起点与编辑位置。

已给：多段目录扫描、日志起点和段关闭路径由 PartitionLog 骨架持有；mutationVersion() 与私有 markMutation() 已提供。

请修改：exercises/src/main/java/io/simplekafka/storage/PartitionLog.java:
deleteBefore(long)。

行为契约与不变量。只删满足条件的封闭段及同 base 的 .index，不删 active，不删除未知文件。返回实际新的 logStartOffset。LEO 不倒退；重开后新起点仍正确。若已有读取位点低于新起点，读取必须明确报范围错误，不自动 seek。发生删除前仅标记一次；no-op 不增加 mutationVersion。

分级提示（不是答案）

一级 从段 base 与下一段 base 确认哪些逻辑 offset 属于可删前缀。

二级 空 active 段与有数据 active 段都要单独考虑，不能把“最后一个段”当成可删项。

三级 删除后关闭并重新打开，再从边界前后分别读取，以验证磁盘文件和内存起点一致。

可观测例子与测试点。构造三个段、保留最后一个活动段并删除旧封闭段后，实际 logStartOffset 移到保留边界；重开仍返回相同起点且 LEO 不变。低于新起点读取报 OFFSET_OUT_OF_RANGE。对应 Step07Test。

常见误区与真实 Kafka 的差异

只删 .log 不删配对索引会制造悬空索引；按阈值删掉 active 会令后续追加失去起点。真实 Kafka 保留常由时间/大小配置与后台管理触发，并可能有 retention 与 consumer 独立关系；课程只提供明确同步调用，不替消费者重置位点。

命令与晋级。 `./gradlew :stepTest -Pstep=7` 用于定位本步；`./gradlew :test -Pstep=7` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么删除阈值通常不会恰好对应某条记录的 segment 边界？
2. 提交位点已经越过旧段，是否意味着系统可以自动删段？为什么？

3.4 第 8 步：尾部截断

目标与前置知识

本步目标：把日志尾部截回某个下一条位点，删除分叉尾数据并恢复可追加状态。

开始前应能解释：理解 half-open 区间、记录边界和段 base。

背景与推理

截断目标 `nextOffset` 表示保留所有 `offset < nextOffset` 的记录；合法范围为 `[logStartOffset, LEO]`。若目标恰在已有段边界，应有可追加的 active segment 以该 base 开始；若位于某段内部，应截断到该记录的物理边界，删除后续 segment，并重建被保留段索引。截到 LEO 是无变化；截到 `logStartOffset` 留下空段，下一条 append 使用这个 offset。截断只改变本地日志尾，不会自动改变复制状态、HW 或消费位点。有效且非 no-op 的截断在首次写入/删除前递增一次本对象 `mutationVersion`，非法与 no-op 不增加。

起点与编辑位置。

已给：段关闭、文件截断、索引重建和全局段表已由前面步骤建立；PartitionLog 骨架提供 `mutationVersion()` 与私有 `markMutation()`，有效截断实现须在首次文件变更前调用。

请修改：`exercises/src/main/java/io/simplekafka/storage/PartitionLog.java:truncateTo(long)`。

行为契约与不变量。只允许 `[logStartOffset, LEO]` 内的 `nextOffset`，超范围拒绝且日志不变。保留 `offset < nextOffset`，所有之后数据及其索引移除；LEO 更新到 `nextOffset`。成功之后可立刻追加且新记录的 offset 正好等于 `nextOffset`。未知文件不可删除。错误参数和截到当前 LEO 不增加 `mutationVersion`；任何有效内容变化均在首个 I/O 前标记。

分级提示（不是答案）

- 一级 将目标理解成“下一条要写的 offset”，而不是要保留的最后一条 offset。
- 二级 分别列出截在段中、截在段边界、截在当前 LEO、截在 `logStartOffset` 的磁盘状态。
- 三级 重开截断后的目录并追加一条；只检查内存 `nextOffset` 不足以证明索引和文件同步。

可观测例子与测试点。写入 offset 0-8 后执行 `truncateTo(5)`, offset 0-4 仍可读; 已删除的 5-8 不会返回, 读取新的 LEO 5 得到空结果, 下一条追加得到 offset 5。`truncateTo(LEO)` 不改变内容; 截到起点后保留空 active 并从该起点追加。对应 `Step08Test`。

常见误区与真实 Kafka 的差异

误将 `nextOffset` 解释为最后保留记录, 会产生 off-by-one; 只 truncate 文件不重建索引, 可能使之后读取跳到删除的分叉记录。真实 Kafka 截断往往与 leader epoch、HW 和副本协议联动; 本步只提供日志原语, 不允许拿它单独推断安全选举。

命令与晋级。`./gradlew :stepTest -Pstep=8` 用于定位本步; `./gradlew :test -Pstep=8` 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿, 才算完成本章并进入下一章。

自检问题

1. 截断到 5 时, 为何下一条应写入 offset 5 而非 4?
2. 为什么清理文件尾之后还必须处理索引文件?

第四章 分区与真实网络 broker

上一章的日志只有本地意义；本章为 topic 建立多个 partition，再用真实 loopback TCP 请求把客户端与 broker 分开。分区决定记录的有序范围，topic metadata 告诉客户端每个 partition 的 leader。TCP 是字节流，不保留 Java write 的边界，因此协议必须自带帧长度。课程只实现教学二进制消息，不兼容 Kafka wire protocol。

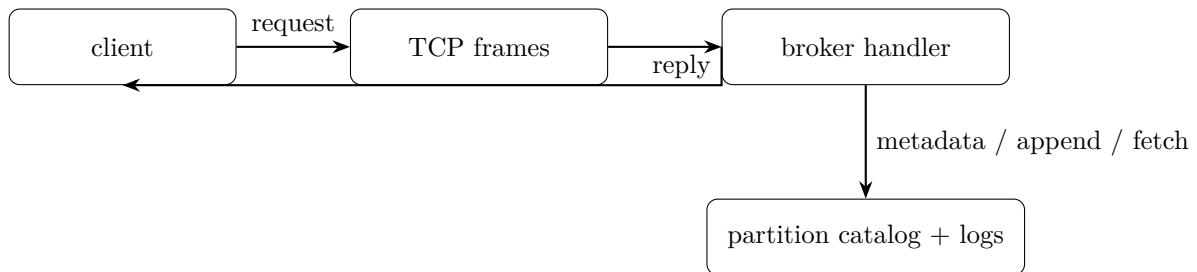


图 4.1: 一个连接上的请求/响应路径。message body 编解码由 provided 代码承担，学生实现 frame 与业务分派。

4.1 第 9 步：分区路由

目标与前置知识

本步目标：让相同 key 稳定进入相同分区，同时为 null key 提供可观察的轮转。

开始前应能解释：理解 key 的字节身份、CRC32C 与取模。

背景与推理

producer 必须先选择 partition 才能确定该分区内的顺序。课程规定 key 非 null 时计算 CRC32C，再把 CRC 视作 unsigned 值对分区数取模；同一字节串因此稳定命中同一 partition。key 为 null 时没有可供 hash 的键，单个 Partitioner 实例按 0 起始轮转。空 byte array 是存在但长度为零的 key，不等价 null。partition count 必须大于零。这个简单规则让输出确定，便于观察 routing 结果。

起点与编辑位置。

已给：partitioner 实例的 round-robin 状态与无参构造器已提供；topic metadata 会给出 partition count。

请修改: `exercises/src/main/java/io/simplekafka/client/Partitioner.java`: `choose` (`byte[]key,intpartitions`)。

行为契约与不变量。 `partitions<=0` 属于非法请求。非 `null key` 路由依赖其 `CRC32C` 的 `unsigned` 解释; 每次选择 `null key` 前, 按当前 `partition count` 归一化实例级游标, 因此 `topic` 数量变化仍会返回有效分区。不同实例各自从零开始。

分级提示 (不是答案)

一级 先区分 `null`、空数组与非空数组三种输入。

二级 检查 Java `signed int` 与题设 `unsigned CRC` 的转换语义, 避免负余数成为负 `partition`。

三级 用同一个实例混合调用有 `key`、`null key` 和不同 `partition count` 的情形, 审视轮转游标是否只在约定输入上推进。

可观测例子与测试点。 3 `partitions` 下, 同一 `key` 连续 100 次选择都相同; 空 `key` 也是确定路由。fresh `Partitioner` 对 `null` 连续六次返回 0、1、2、0、1、2。 `partition count` 为 3、3、1、3、3、2、2 时, `null-key` 选择为 0、1、0、0、1、0、1。对应 `Step09Test`。

常见误区与真实 *Kafka* 的差异

把 `key` 的 `hashCode` 当作 `CRC32C`, 或对可能负值直接取余, 都会违反路由契约。真实 *Kafka* 默认 `partitioner` 的 `key hash` 算法和 `null-key sticky batching` 策略与本课程不同; 此处 `null key` 采用实例级 `round-robin`, 不应描述成 *Kafka* 默认行为。

命令与晋级。 `./gradlew :stepTest -Pstep=9` 用于定位本步; `./gradlew :test -Pstep=9` 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿, 才算完成本章并进入下一章。

自检问题

1. 为什么 `hash` 应作用于 `key bytes` 而不是 `RecordData` 对象的 Java `hashCode`?
2. `key` 为 `null` 与 `key` 为零长度数组, 在协议和路由上有何不同?

4.2 第 10 步: 主题元数据

目标与前置知识

本步目标: 在安全目录结构中登记 `topic` 的固定 `partition` 集合, 并让客户端按确定顺序发现 `leader`。

开始前应能解释: 了解分区元数据和幂等创建的意义。

背景与推理

元数据把逻辑名称映射到物理分区。每个分区至少携带 TopicPartition、leader id、epoch、replica/ISR 列表和 Endpoint；单机模式的 epoch 为 0，replicas 与 ISR 都只含当前 broker。一个 topic 的 partition 数在这个课程 catalog 中创建时固定。目录结构为 root/topic/partition，topic 名受 `[A-Za-z0-9._-]+` 限制并拒绝 `.` 与 `..`，避免路径逃逸。目录重启时显式再次创建相同 topic/count 来打开既有日志，不从偶然存在的文件猜测分区总数。

起点与编辑位置。

已给：endpoint、分区元数据值类型与日志注册路径已提供；Catalog 构造器固定 broker id 和 endpoint。

请修改：exercises/src/main/java/io/simplekafka/broker/PartitionCatalog.java：
createTopic(String,int)、metadata(String)。

行为契约与不变量。正数 partition count 创建固定分区；同 topic 同 count 幂等，不同 count 报 INVALID_REQUEST。未知 topic 报 UNKNOWN_TOPIC_OR_PARTITION。metadata 按 partition id 递增返回，不泄露 root 外路径。单机 metadata 的 epoch=0、replicas/isr=[brokerId]。

分级提示（不是答案）

- 一级 创建 topic 不只是 mkdir：要考虑 partition 数和后续重启时可验证的约束。
- 二级 让同名同数量调用和同名不同数量调用走不同语义。
- 三级 用特殊路径字符、空名和目录重开做行为测试，确认名字验证在文件创建前发生。

可观测例子与测试点。创建 orders 2 partitions 后，metadata 中包含 orders-0、orders-1 且顺序固定；再次创建相同定义无损；改为 3 报错；未知主题查询给出 topic/partition 错误码。对应 Step10Test。

常见误区与真实 Kafka 的差异

直接把用户 topic 拼进 path 会让路径穿越风险进入磁盘边界；靠目录枚举恢复 partition 数会把手工残留当元数据。真实 Kafka topic 元数据由集群控制面管理，partition 可在受控操作中扩展；本课程 catalog 只做显式固定数量，不处理动态扩容。

命令与晋级。 ./gradlew :stepTest -Pstep=10 用于定位本步；./gradlew :test -Pstep=10 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为何“同名但不同分区数”不能当成幂等成功？
2. 磁盘目录能提供哪些证据，不能替代哪些元数据？

4.3 第 11 步：TCP 帧协议

目标与前置知识

本步目标：把 TCP 字节流切分成有界、可匹配的请求/响应帧。

开始前应能解释：理解 `readFully`、EOF 位置、长度前缀和 `correlation id`。

起点与编辑位置。

已给：Messages 请求/成功响应 `body`、API `id` 常量、Frame 值类型与 `ErrorCode` `wire id` 已提供；`body codec` 不由本步重写。

请修改：`exercises/src/main/java/io/simplekafka/protocol/FrameCodec.java`：
`write(OutputStream, Frame)`、`read(InputStream)`。

行为契约与不变量。第一字节前 EOF 返回 `null`；帧中途 EOF 为 `INVALID_REQUEST`。拒绝负数或超限长度后再分配。未知 `version/api` 和非零请求 `error` 由服务端验证拒绝。多帧连续流逐帧读取不丢边界；输出一帧不能发半帧。Stream I/O 按接口抛 `IOException`，协议格式错误使用课程错误码。响应 `body` 超出单帧上限时，服务端返回有界的 `INVALID_REQUEST` `error Reply`，而不是静默关闭连接。

分级提示（不是答案）

一级 帧的最短长度由固定 `header` 各字段宽度之和决定，不由示例 `payload` 推测。

二级 把“干净的流结束”与“帧内提前结束”分开处理。

三级 用每次只返回一个 `byte` 的 `InputStream` 喂入连续两帧，并用只有超长前缀的输入检查是否立即拒绝。

可观测例子与测试点。把两帧流拆成单字节块，连续两次 `read` 得到原 `correlation id` 和 `payload`；下一次 `read` 才得到 `null`。只有 4-byte 超长 `length`、没有 `body` 的输入会立即失败而不是等待或分配数 MB。对应 `Step11Test`。

常见误区与真实 Kafka 的差异

使用一次 `read` 期待填满 `header` 会受到 TCP 分段影响；先分配后检查 `length` 会被恶意请求放大内存。真实 Kafka 使用自己的请求头、API 版本协商与消息格式，本课程 `version=1/API` 列表是教学协议，不可接入官方客户端。

命令与晋级。`./gradlew :stepTest -Pstep=11` 用于定位本步；`./gradlew :test -Pstep=11` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么 `correlationId` 必须被回送给对应请求？
2. 在流中区分“正常 EOF”和“半帧 EOF”有什么价值？

4.4 第 12 步：单机 broker

目标与前置知识

本步目标：把 metadata、produce、fetch 三种业务请求接入本地 catalog，并通过真实 TCP 端到端验证。

开始前应能解释：完成 frame、body codec 和单段/分段日志步骤。

背景与推理

broker 是请求语义边界：收到请求先验证 topic、partition、epoch 与参数，再调用日志，最后把成功结果或统一错误包装成 Reply。单机模式把 log end offset 同时视为 high watermark，因此 append 后消费者可立即读。produce 返回追加区间，fetch 按偏移和两种预算读取记录并返回 logStart、LEO、HW 与 epoch。未知 api 在 transport 层拒绝；业务错误只影响本次请求，broker 不应因此停止接受后续 TCP 请求。真实 TCP 调用能同时验证客户端 framing 与服务端 handler，而不只是直接调用方法。

起点与编辑位置。

已给：BrokerHarness.single 管理日志注册、TCP transport、MessageCodec、请求体解析与服务生命周期；SingleBrokerMain 使用 loopback 和 OS 分配端口。

请修改：exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java：handle(Request)。

行为契约与不变量。先实现 METADATA/PRODUCE/FETCH，未知分区错误为 UNKNOWN_TOPIC_OR_PARTITION，单机 epoch 仅接受 0，位点越界保留对应错误码。成功 produce 返回 AppendResult；成功 fetch 不越过 LEO=HW。错误 Reply 只带错误码和简短 message，不把堆栈送上线。课程 RpcClient 每次请求使用新 TCP 连接；一次业务错误不得让 broker 停止接受后续请求。

分级提示（不是答案）

一级 按 API 类型分派，各请求在访问日志之前先完成身份和范围校验。

二级 检查 error 发生在 transport、handler 还是日志层；不同层的责任不可互相吞并。

三级 用一条真实 loopback 连接先 produce、fetch，再发送一个失败请求并继续发送合法请求。

可观测例子与测试点。经 TCP produce 三条记录，返回首尾区间；fetch 同一 partition 得到三个 offset/key/value 及 logStart=0、LEO=HW=3。unknown partition 返回正确错误码，随后通过一个新的 TCP 请求合法 fetch 仍成功。对应 Step12Test，命令行也可用 SingleBrokerMain 与 ClientMain 手动观察。

常见误区与真实 Kafka 的差异

仅调用 handler 单元方法无法发现 frame 边界/correlation id 错误；把异常转成空成功响应会隐藏错误且破坏客户端语义。真实 Kafka broker 还有认证、配额、请求版本兼容、持久控制面

等；本课程 broker 绑定 loopback，API 与官方客户端不兼容。

命令与晋级。 `./gradlew :stepTest -Pstep=12` 用于定位本步；`./gradlew :test -Pstep=12` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么 broker 错误回复不应泄露 Java 堆栈？
2. 单机 HW=LEO 为何不代表副本模式下也能这样处理？

第五章 生产消费、位点与重复投递

本章把直接调用 broker 的教学程序变成可理解的客户端行为。producer 将消息按分区缓冲，consumer 按显式分配拉取，OffsetStore 把已处理进度写入本地日志。关键区分是三个位置：broker 的 LEO、复制模式中的 HW，以及某个 group 的 committed offset。一次拉取推进客户端内存 position，不等于 commit；commit 也不等于 broker 日志数据已经删除。客户端接管直接传入的 RpcClient，并在 close 时关闭它；传入的 MetadataRouter 是借用的共享依赖，应由创建者在使用它的客户端全部关闭后统一关闭。

5.1 第 13 步：批量生产者

目标与前置知识

本步目标：实现按分区有序的批量缓冲、阈值发送和显式 flush。

开始前应能解释：理解 metadata 路由、batch 的边界、返回 receipt 与请求结果不确定性。

背景与推理

每个分区的 batch 必须维持调用顺序，但不同分区可以分别积攒。send 根据 metadata 得到 partition，并在达到 batchRecords 时同步发送该分区 batch；flush 按稳定顺序发送所有非空 batch 并汇总 ProduceReceipt。自动发送返回的 receipt 不能丢失，须留到下一次 flush 交给调用者。默认 ACK 为 LEADER，不使用后台定时器。produce RPC/回复处理中的 timeout 或错误都会使结果未知：broker 可能已经追加。该路径开始后的任何失败都会进入“结果未知”状态，后续操作直到关闭前均失败，不猜测并重放。

起点与编辑位置。

已给：RpcClient 管理 socket、timeout、correlation id 和请求/响应；metadata、Partitioner 和批次记录结构已提供。

请修改：exercises/src/main/java/io/simplekafka/client/SimpleProducer.java：
send(String,byte[],byte[],long)、flush()。

行为契约与不变量。相同 partition 保持 send 顺序，分区间 batch 可独立；batch 满时同步发送该分区。receipt 保存 topic-partition 与 [firstOffset,nextOffset)；显式 flush 返回自动发送和本次发送的 receipt。produce RPC/回复处理一旦开始，任何失败都使追加结果未知：保留首次错误且禁止重放 batch。此后 send、flush 或 setAcks 均报 REQUEST_TIMEOUT，直到关闭 producer。metadata 查询或请求准备阶段、进入该路径前的失败不会 poison producer。

分级提示（不是答案）

- 一级 先明确“发送请求”“得到确认”“向调用者交还 receipt”是三个不同时间点。
- 二级 用 TopicPartition 作为缓冲键，检查 flush 顺序和空 batch 行为。
- 三级 让 broker 追加后再返回错误，或延迟响应直到 timeout；验证客户端不会再产生第二份数据。

可观测例子与测试点。配置 3 partitions 和 batchRecords=2，连续发送同 key 数据应路由至同一分区并保序；null key 分散按实例轮转。达到两条时发送一次，调用 flush 可取得包含自动发送 receipt 的结果。追加后返回错误时保留首次错误；后续 send、flush、setAcks 报 REQUEST_TIMEOUT 且不重放。对应 Step13Test 与 Step27Test。

常见误区与真实 Kafka 的差异

若自动 flush 的 receipt 被丢弃，调用者无法知道已经成功的 offset 区间；若把 RPC/回复错误当作“肯定未追加”并重试，可能重复。本章的 SimpleProducer 刻意同步发送且不自动重试。第 8 章新增独立的 IdempotentProducer，调用方可用稳定 producer identity 显式重试 batch；这不会改变普通 producer 的结果未知契约。

命令与晋级。 ./gradlew :stepTest -Pstep=13 用于定位本步；./gradlew :test -Pstep=13 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为何 timeout 不足以证明 broker 没有追加？
2. 如果同一个 batch 的错误回复被重试，调用方如何识别重复？本课程为何不做这个机制？

5.2 第 14 步：拉取消费者

目标与前置知识

本步目标：构建单线程、单轮 pull consumer，明确管理 assigned partitions 与本地 position。开始前应能解释：理解 seek、空 fetch 与总预算。

背景与推理

consumer 的 position 指下一条要处理/拉取的 offset。手动 assign 对分区去重并排序，用新 map 替换旧 assignment：仍保留的分区继续使用原 position，撤销分区被删除，新分区初始 position=0；它不读取 OffsetStore。seek 可重放或跳转但不提交。poll 是一次有限请求，不是持续长轮询；跨分区共享 maxRecords/maxBytes 总预算。成功返回记录后，仅相应分区 position 移到最后一条 offset+1；某分区 fetch 失败不能提前推进它的 position。空结果意味着当前位置暂时没有可读数据，不代表应该向前跳。consumer 按单线程使用，避免并发 poll 对 position 产生竞争。

起点与编辑位置。

已给：RpcClient 已提供 FETCH；SimpleConsumer 持有 group 名和分区 position，并提供只读 position(tp)。第 14 步不调用 OffsetStore。

请修改：exercises/src/main/java/io/simplekafka/client/SimpleConsumer.java：assign(Collection)、seek(TopicPartition,long)、poll(int,int)。

行为契约与不变量。 assign 去重并稳定排序；仍保留的分区 position 不变，新分区从 0 初始化，撤销项移除，不读取 committed offset；seek 目标分区必须已分配，否则 NOT_ASSIGNED。poll 成功后只推进返回记录所在分区；空 fetch 不变；失败分区不前移。读不自动 commit。总 records/bytes 不超过请求限额；broker 会把过大的正 maxBytes 限制到保守的单帧预算，后续 fetch 从首个未返回 offset 继续。返回映射按可预期分区顺序组织。

分级提示（不是答案）

- 一级** 将“已分配”“有 position”“有 committed offset”分别建模，不要在早期手动模式混入 store。
- 二级** 读取各分区时维护同一个 poll 剩余预算，而不是为每个分区重置。
- 三级** 让两个分区中一个请求失败，检查另一个可成功分区和失败分区的 position 分别如何变化。

可观测例子与测试点。 assign 两个 partition 后初始 position 均为 0；poll 读出 offset 0、1 后 position=2；seek 到 0 再 poll 能重读 0。空 fetch 不动 position，未分配分区 seek 报 NOT_ASSIGNED。对应 Step14Test。

常见误区与真实 Kafka 的差异

poll 返回记录不等于已经持久提交；seek 也只是本地 position 改动。每个分区都用满完整 maxBytes 会造成总预算超标。真实 Kafka consumer 还提供长轮询、线程限制与复杂的 assignment/position 生命周期；本课程一步一轮且明确使用单线程。

命令与晋级。 ./gradlew :stepTest -Pstep=14 用于定位本步；./gradlew :test -Pstep=14 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么读取成功后推进 position 仍不代表重启后从新位置开始？
2. 某分区请求失败时，为何不能把所有分区 position 一起推进？

5.3 第 15 步：消费位点持久化

目标与前置知识

本步目标：持久化下一条要处理的 offset，并在新 consumer 中恢复。

开始前应能解释：区分物理日志 offset、group/tp 键与消息协议。

背景与推理

OffsetStore 是一个追加日志：每次 commit 将 (group,TopicPartition,nextOffset) 编码进一条 LogRecord，物理日志自身分配递增 offset，记录 payload 中保存的 nextOffset 才是消费进度。key=null、timestamp=0；provided MessageCodec 负责编码 OffsetKey/value，重启扫描同 key 最新值。底层使用第一章记录格式与 force，损坏尾部遵循相同恢复规则。consumer.resume 先做 assign，再分别查 committed value；没有已提交值时从 0 开始。消费位点是下一条 offset，因此提交 3 后恢复从 3 拉取。COMMIT_OFFSET/FETCH_OFFSET 请求也由 broker handler 统一路由。

起点与编辑位置。

已给：MessageCodec 的 offset entry 编码、RpcClient、consumer 分配和日志记录工具已提供。

请修改：exercises/src/main/java/io/simplekafka/group/OffsetStore.java: commit (OffsetKey,long)、fetch(OffsetKey)；exercises/src/main/java/io/simplekafka/client/SimpleConsumer.java: resume(Collection)、commitSync()；并增补 BrokerHandler 的 COMMIT_OFFSET/FETCH_OFFSET。

行为契约与不变量。 offset 是下一条记录编号，可显式回退；不同 group 与 TopicPartition 独立。commit 后重开 store 仍得到最新值；缺少值是空 OptionalLong。resume 在未提交时 position=0，成功 fetch 后可 commitSync 所有 assigned position。组 token 到第 20 步才用于提交围栏，手动消费传 null。

分级提示（不是答案）

一级 先把消费进度定义为值，而不是物理日志记录编号。

二级 扫描同一 OffsetKey 时需要明确多次 commit 哪一条是最终值；不要假定 map 顺序。

三级 测试既要重启 store，也要重建 consumer，分别验证磁盘持久化和 position 初始化。

可观测例子与测试点。 读取 offset 0、1、2 后提交 nextOffset=3；关闭重开 OffsetStore，fetch 得 3；新 consumer.resume 从 3 开始，而另一 group 仍从 0 开始。物理 OffsetStore 日志自己的记录 offset 与 payload 中 3 不要求相等。对应 Step15Test。

常见误区与真实 Kafka 的差异

把最后处理的 offset (2) 直接提交，会让重启再次读 2；把 group offset 与 HW 混为一谈，会错误地认为 commit 使数据可见或不可见。真实 Kafka 将 offsets 存在复制/压缩的内部主题 __consumer_offsets，而本课程是一个本地单 coordinator 追加日志，不提供 HA。

命令与晋级。 ./gradlew :stepTest -Pstep=15 用于定位本步；./gradlew :test -Pstep=15 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么消费位点约定为 next offset 比 last processed offset 更便于表达?
2. HW 与 group committed offset 分别控制什么?

5.4 第 16 步：至少一次处理

目标与前置知识

本步目标：建立处理副作用与 offset 提交的安全先后顺序，并能解释重投。

开始前应能解释：理解失败重启、批次位置和提交原子性边界。

背景与推理

本课程的 ProcessingLoop 单轮 poll 后按 TopicPartition 排序、同分区 offset 顺序执行 processor。只有这一轮所有被返回记录都处理成功，才统一 commitSync；processor 抛异常就传播原异常，不提交任何分区。consumer 的内存 position 已经随 poll 推进，所以失败实例不能继续使用，否则会跳过尚未 commit 的记录；应关闭并重建，从已持久化位置恢复。副作用与 offset commit 并非一个原子事务：若副作用已发生但 commit 尚未成功，重启后记录会再次处理。于是至少一次容许重复，但不应静默丢失未成功处理的记录。

起点与编辑位置。

已给：RecordProcessor 回调接口、单轮 poll、commitSync 与重建所需的 consumer 生命周期已提供。

请修改：exercises/src/main/java/io/simplekafka/client/ProcessingLoop.java:runOnce(int,int)。

行为契约与不变量。返回成功处理的记录数。记录处理按稳定分区顺序与 offset 顺序调用；全部成功后才提交。任一 processor 异常原样传播且本轮不提交任何分区。失败后调用者应关闭这个 consumer 并从 store 重建，禁止将 poll 已推进 position 当作已持久进度。

分级提示（不是答案）

- 一级 写出“poll、外部副作用、commit、崩溃”的时间线，并找出唯一可出现重复的窗口。
- 二级 判断跨多个分区批次时，提交部分分区会不会违反本题全部成功才 commit 的契约。
- 三级 分别在第 1 条处理失败与所有处理成功之后重启，比较观察到的投递序列。

可观测例子与测试点。处理 offset 0 成功，处理 offset 1 抛异常；重建 consumer 后再次获得 0、1。两条均成功处理并提交后再重建，不会再次投递。测试记录第一次副作用并断言 0/1 的重放与最终 offset；对应 Step16Test。

常见误区与真实 Kafka 的差异

把处理函数抛错吞掉再 commit 会产生数据丢失；继续使用失败实例会跳过重放。至少一次并不等于 exactly-once，broker 也不会自动让外部副作用幂等。第 8 章说明应用如何通过稳定业务 event id 使本地持久化效果幂等；但不会让副作用和 offset commit 变成一个事务。课程没有事务，也不把本地 OffsetStore 说成事务边界。

命令与晋级。 `./gradlew :stepTest -Pstep=16` 用于定位本步；`./gradlew :test -Pstep=16` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 在副作用成功与 offset 持久化之间崩溃，为什么出现重复是合理的？
2. 要避免继续运行失败 consumer，关键是内存 position 还是持久化 offset？为什么？

第六章 消费组、重平衡与提交围栏

消费组让多个 consumer 协作读取一个 topic：每个 partition 同一时刻分给一个成员，成员增减会改变分配。仅有“谁读取哪个分区”还不够；旧成员可能在重平衡后继续提交旧进度，因此必须用 generation token 围栏。课程刻意做一个单 topic 的即时重分配模型，不是完整 Kafka classic 协议或新式 consumer group protocol。Coordinator 是本机可信状态，不具备 HA。

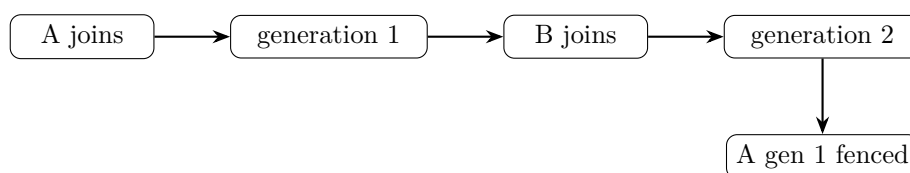


图 6.1: 成员拓扑变化递增 generation；旧 token 不能覆盖当前提交。

6.1 第 17 步：分区分配

目标与前置知识

本步目标：实现确定、平衡、每分区唯一 owner 的 Round Robin 分配。

开始前应能解释：掌握集合去重、稳定排序和商余数分布。

背景与推理

如果输入集合顺序由网络、HashMap 或启动顺序决定，同一组成员可能得到不稳定分配。课程先对 TopicPartition 按 topic/partition 排序、对 member 按字符串字典序排序，再把第 i 个分区交给 $\text{member}[i \bmod n]$ 。因此分配可复现，且分区数在成员间差最多为 1。所有已知成员都要出现在结果中，即使其分区列表为空；无成员时返回空 map。共享 topic 前提由 coordinator 检查，assignor 只处理给定的分区和成员。

起点与编辑位置。

已给：TopicPartition 值类型与 group coordinator 依赖接口已提供；调度算法只有一个单独 assignor。

请修改：exercises/src/main/java/io/simplekafka/group/RoundRobinAssignor.java：
`assign(List<TopicPartition>, List<String>)`。

行为契约与不变量。先去重并按固定顺序排列输入。每个不同 partition 恰出现一次，owner 必须属于 members；member 的空列表仍保留。成员为空时结果为空。每个成员分配数量的最大值与最小值之差不大于 1。

分级提示（不是答案）

- 一级 把唯一 owner 属性转成总覆盖与无重复两个可检查条件。
- 二级 要先明确 member 顺序；输入 List 的原始顺序不等于课程的稳定顺序。
- 三级 手算 5 个分区、2 个成员和 5 个分区、7 个成员，再测试重复输入不让分区多分一次。

可观测例子与测试点。五个排序后的 partition 与成员 A、B 得 A 三个、B 两个；七成员分五分区时五人各一个、两人空列表。打乱输入顺序仍产生同一结果。对应 Step17Test。

常见误区与真实 Kafka 的差异

只返回有分区的成员会丢失明确空分配；在遍历 map 时按偶然顺序分配则难以测试和复现。真实 Kafka 有多种 assignor、静态成员、协作重平衡等；课程只实现单 topic 的简单 Round Robin，不能把它等同 Kafka 默认或完整协议。

命令与晋级。 ./gradlew :stepTest -Pstep=17 用于定位本步；./gradlew :test -Pstep=17 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为何稳定排序是可观察行为的一部分，而不只是美化输出？
2. 成员比分区多时，空分配如何帮助调用者表达组状态？

6.2 第 18 步：成员与重平衡

目标与前置知识

本步目标：管理 group 成员集合与 generation，并在拓扑改变时更新全体分配。

开始前应能解释：理解 generation 不是时间戳、也不是消费位点。

背景与推理

组内成员订阅同一 topic。新成员加入或已有成员离开，assignment 发生变化，generation 加一；一个组从 generation 0 开始。重复 join 同一 member 且 topic 不变只刷新心跳，不产生新 generation；改订阅 topic 在这个简化模型中非法。每次变化要用新的成员集合重新计算完整分配，不能只给加入者临时分区而忘记撤销其他成员。组变空时仍保留 generation 与 committed offsets，避免后续组状态和消费进度混淆。

起点与编辑位置。

已给：组锁、成员状态容器、TimeSource、assignor 接口与 generation 生命周期由构造器提供。

请修改：exercises/src/main/java/io/simplekafka/group/GroupCoordinator.java：
join(String,String,String)、leave(String,String)、assignment(String,String)；
handler 增补对应 API。

行为契约与不变量。首次加入 generation=1；拓扑真实变化时恰递增一次并返回最新全组 assignment。重复相同 join 不递增；topic 不同报 INVALID_REQUEST。unknown member leave/assignment 报 UNKNOWN_MEMBER。组清空后状态及 offset 保留。

分级提示（不是答案）

- 一级** 先找出会改变成员集合的事件，再区分重复请求与新状态。
- 二级** 重平衡后每个仍在组内的成员都要看到相同 generation 的全组映射。
- 三级** 按照 A 加入、B 加入、A 离开的时序手工列出 generation 和 owner，再检查 empty group 之后的再加入。

可观测例子与测试点。A 首次加入得 generation 1；B 加入后为 2 且两人分摊全部 partition；A 离开后为 3，B 获得所有 partition。A 同 topic 重复 join 不再增加 generation。对应 Step18Test；JOIN、ASSIGNMENT、LEAVE 均经真实 TCP broker API 检查。

常见误区与真实 Kafka 的差异

把每个 member 自己的分配版本单独推进，会使组内 token 对不上；组为空就销毁 offset store，会让重启进度丢失。真实 Kafka 在加入、同步 assignment、cooperative revocation 等阶段有完整的协调协议；课程直接即时重分配，不模拟完整 classic barrier 或 KIP-848。

命令与晋级。 ./gradlew :stepTest -Pstep=18 用于定位本步；./gradlew :test -Pstep=18 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么 group generation 变化要让未离开的成员也收到新 assignment？
2. 组没有成员时，为什么仍可以保留消费进度？

6.3 第 19 步：心跳与过期

目标与前置知识

本步目标：在可控时间下检测成员超时，并把同组过期成员作为一个拓扑变化原子处理。
开始前应能解释：理解心跳时间与过期阈值。

背景与推理

每个 member 保存 lastHeartbeat, 心跳成功刷新它。若 `now-lastHeartbeat >= sessionTimeoutMillis`, 成员到期。时间来自注入的 `TimeSource`, 因此边界测试不需要真实 sleep。一次 expire 可能同时发现多个过期成员; 同一 group 应一次移除所有到期成员, 只增加一次 generation 并重分配一次, 避免一次心跳 tick 产生无意义的多代状态。先检查未知 member; 旧 generation 错误也不能延长生命期。单机与复制 harness 都运行周期过期 worker; 复制 harness 还提供 `expireGroups()` 供确定性测试使用。

起点与编辑位置。

已给: 注入式 `TimeSource`、组锁、session timeout 与调度生命周期已提供; 测试可用确定性 clock。

请修改: `exercises/src/main/java/io/simplekafka/group/GroupCoordinator.java`: `heartbeat(GroupToken)`、`expire()`; `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java`: `HEARTBEAT` 分支。

行为契约与不变量。 边界采用 `>=`。有效当前 token 的 heartbeat 更新时间; 未知成员报 `UNKNOWN_MEMBER`, generation 过期报 `ILLEGAL_GENERATION`, 失败不刷新。expire 返回被移除的 group/member 标识; 同组多成员到期一次只增加一次 generation。leader/partition 概念不进入此状态。

分级提示 (不是答案)

- 一级 把时钟读取、超时判断和成员移除放在一致的状态观察中。
- 二级 返回值应能精确区分被哪个 group 删除的 member, 而非只有数量。
- 三级 用 1099 与 1100 两个时刻测试 `timeout=100` 的边界, 并让 B 刷新后与 A 同时比较。

可观测例子与测试点。 初始时间 1000、`timeout=100`, 时刻 1099 时 A 保留, 1100 时 A 删除; B 在 1099 心跳后到 1100 仍保留。若 A 与 C 同组同一时刻过期, generation 只增一次。对应 `Step19Test`: Heartbeat 使用真实 TCP, 边界由确定性 hook 验证, 另有复制集群场景验证 worker 会在无心跳时过期成员。

常见误区与真实 Kafka 的差异

用真实系统时间和 sleep 做精确边界测试会受调度抖动影响; 边界断言应使用注入 clock 和显式过期 hook, 并单独保留周期 worker 的集成验证。未验证 token 就先改 lastHeartbeat 会让旧 member 非法续命。真实 Kafka 的 session timeout/heartbeat 还受协议、消费者配置与网络调度影响。

命令与晋级。 `./gradlew :stepTest -Pstep=19` 用于定位本步; `./gradlew :test -Pstep=19` 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿, 才算完成本章并进入下一章。

自检问题

1. 为什么过期判断的等号边界必须写进契约？
2. 一批同组成员在同一次 expire 过期，为何不应多次 generation++？

6.4 第 20 步：消费组围栏

目标与前置知识

本步目标：把 group assignment 和 offset 操作结合，通过 generation token 拒绝僵尸成员写入。

开始前应能解释：理解 token 的组/member/generation 三元组与撤销分区。

背景与推理

GroupToken 由 group、member 和 generation 构成。FETCH 与 COMMIT 带 token 时，coordinator 必须验证成员存在、generation 当前、partition 当前分配给该成员。generation 已变的旧 consumer 不能提交；验证和 OffsetStore.commit 必须处在 coordinator 锁的原子临界区，锁顺序为 coordinator 到 store，不持有 partition 锁。GroupConsumer 每次 poll 先 heartbeat。若遇 ILLEGAL_GENERATION，它查询当前 assignment、丢弃已撤销 partition 的旧 position、对新分区从 committed offset resume，再用新 token fetch；UNKNOWN_MEMBER 不自动重新加入，需显式 subscribe。manual token=null 保持手动模式，不伪装为成员。

起点与编辑位置。

已给：group 消息、RpcClient、SimpleConsumer 拉取/位点路径和成员生命周期骨架已给出；broker 带 token 的请求体已编码。

请修改：exercises/src/main/java/io/simplekafka/group/GroupCoordinator.java: validate(GroupToken, TopicPartition); exercises/src/main/java/io/simplekafka/client/GroupConsumer.java: subscribe(String)、poll(int, int)、commitSync(); 同步增补 handler 围栏。

行为契约与不变量。当前 generation 以外 token 报 ILLEGAL_GENERATION；未知 member 报 UNKNOWN_MEMBER；未分配分区报 NOT_ASSIGNED。失败不更改 store。新 assignment 生效后旧撤销 position 不能提交。commit validate 与追加在一个 coordinator 锁范围完成。服务端不保证撤回已发出的旧成员缓存记录，只保证提交围栏。

分级提示（不是答案）

- 一级 验证请求要同时检查 token 身份、当前 generation、partition owner，不能只比较 group 名。
- 二级 过期处理后 consumer 的旧 position 哪些可以保留、哪些必须从 committed offset 重建？
- 三级 制造两个真实 TCP consumer，让旧成员以旧 token 提交较大位点，再检查失败后 store 的值。

可观测例子与测试点。 A generation 1 加入, B 加入使 generation 2; A 用旧 token 提交 100 被拒, B 以有效 token 提交 7 后 store 仍为 7。真实双 consumer 分摊 4 分区, 成员过期后另一成员接管并从已提交位点继续。对应 Step20Test。

常见误区与真实 Kafka 的差异

validation 后释放锁再 commit 会让重平衡插入其间, 僵尸提交覆盖有效进度。把 ILLEGAL_GENERATION 和 UNKNOWN_MEMBER 都自动处理成重新 join 会隐藏身份错误。真实 Kafka 组协议涉及同步屏障和多个阶段, 事务提交语义另有约束; 课程的单 topic coordinator、共享 JVM authority 与本地 OffsetStore 均不是高可用 __consumer_offsets。

命令与晋级。 ./gradlew :stepTest -Pstep=20 用于定位本步; ./gradlew :test -Pstep=20 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿, 才算完成本章并进入下一章。

自检问题

1. generation fencing 能保证什么, 又为何不能保证外部副作用 exactly-once?
2. validate 与 offset append 若分开持锁, 中间可能发生什么竞争?

第七章 拉取副本、ISR、高水位与确认

本章从单机日志进入复制。**课程的分布式模型边界必须先读清：**三个独立 broker 各有 TCP server、线程与磁盘目录，follower 必须经 TCP 拉记录，不能共享内存复制；但它们运行在同一个 JVM，受一个可信 ClusterAuthority 管理。authority 线性化 metadata、epoch 与 HW 状态，保证教学切换可控。它不是 KRaft，不具备 controller 共识或 authority HA；整个 JVM/authority 丢失后的集群恢复不作保证。

复制只有在追加、报告和可见性边界彼此一致时才有意义。LEO 是每份日志自己的下一条 offset。ISR 是当前被允许参与确认的副本集合。HW 是普通消费者能读取的提交边界，消费者只能读取 $\text{offset} < \text{HW}$ 。副本通过真实网络拉取；authority 只提供一致状态和角色决策，不代替数据移动。

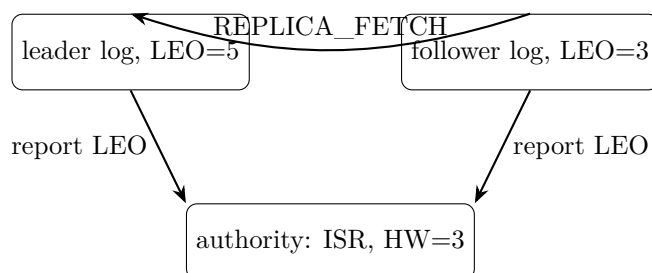


图 7.1: 副本日志由 TCP 拉取，进度由可信 authority 串行化。示例 $\text{HW} = \min(5, 3) = 3$ 。

7.1 第 21 步：拉取复制

目标与前置知识

本步目标：让 follower 通过 REPLICATION_FETCH 请求把 leader 的记录持久化到本地。

开始前应能解释：理解 leader LEO、follower LEO 和读请求为何不同于消费者 fetch。

背景与推理

FollowerReplicator 在第一次 state monitor 临界区验证 online/follower 并捕获 epoch，随后完全在锁外执行 REPLICATION_FETCH。leader 返回 records、epoch、HW、leader LEO；复制可读到当前 leader LEO，不受普通消费者 HW 限制。follower 检查响应 epoch、首 offset、连续性与 append offset 对齐，再进入第二次短临界区，先复核 epoch，再复核 online/follower，并在同一临界区完

成空 poll 或日志 append。provided ReplicaState.update 同步发布角色，避免末次检查与 append 间切换角色；网络调用不持 state monitor。只有本地 append 持久化成功后，下一轮 poll 才能报告进度。第 21 步网络只读写日志，不调用尚未实现的 report。

起点与编辑位置。

已给：三节点 TCP harness、REPLICA_FETCH 消息类型、ReplicaState 与 PartitionLog 生命周期已提供；fixture seed 会把 leader 实际 LEO 放入初始进度。

请修改：exercises/src/main/java/io/simplekafka/replication/FollowerReplicator.java: pollOnce(int,int)；同时增补 BrokerHandler 的 REPLICA_FETCH 分支。

行为契约与不变量。仅当前 online follower 可复制正确分区；发送请求前捕获 epoch，返回后必须在网络锁外确认响应格式，再在本地 state monitor 内复核当前 epoch 与角色。期间 epoch 改变报 FENCED_EPOCH，角色变为 leader/offline 报 NOT_LEADER；这些失败不能追加。broker 会把过大的正 maxBytes 限制到单帧响应预算；有界批次可在多次 poll 中保持 follower offset 连续。重复空 poll 不重写。第 21 步只验证副本数据一致，不报告进度或推进 HW。

分级提示（不是答案）

一级 把“收到网络响应”与“本地持久化完成”分成两阶段观察。

二级 记录响应应包含多项边界信息：分别理解 HW 与 leader LEO 的不同用途。

三级 让 follower 重复 poll、让响应 epoch 错误、让第一条 offset 不符，确认只有有效 batch 改变本地 LEO。

可观测例子与测试点。leader 含 offset 0-2，follower 起点为 0；一次 poll 后 follower 磁盘读取到同样 key/value 与 offset 0、1、2，第二次空 poll 返回 0 条且 LEO 不变。epoch 错误不能改 follower。大回复会限制在单帧预算内，并由后续 poll 连续复制。对应 Step21Test 的真实 TCP 与文件断言。

常见误区与真实 Kafka 的差异

直接共享 leader 的 PartitionLog 对象会绕过 TCP 和本地落盘，测试就不再评估复制。把 HW 当成 leader LEO 会阻止 follower 追赶；把收到 bytes 当成功则忽略本地写失败。真实 Kafka 有批量复制请求和更多 epoch/状态交互；课程自定义 REPLICA_FETCH 是教学 API。

命令与晋级。./gradlew :stepTest -Pstep=21 用于定位本步；./gradlew :test -Pstep=21 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为何 follower 复制可以读取超过普通消费者可见边界的记录？
2. 为什么本地 append 完成前不能报告 follower 新 LEO？

7.2 第 22 步：ISR 与高水位

目标与前置知识

本步目标：用副本进度报告维护 ISR 和 HW，并用受控时间剔除长期落后的副本。

开始前应能解释：理解 minISR、LEO 单调性与高水位公式。

背景与推理

每份副本进度按 partition 锁保护。当前 epoch 合法的 assigned/online 副本才能 report 非负 LEO。leader 自身报告推进 leaderLEO；follower 报告不得超过 leader 当前 LEO，也不得在同 epoch 倒退。follower 只有追到当时 leader LEO 才刷新 caught-up 时间；仅连接在线不足以证明同步。超时副本从 ISR 移除但不移 leader。ISR 包含 leader 且大小至少 minISR 时，候选 HW 为 ISR 最近成功报告 LEO 的最小值；HW 只能单调推进，达到 minISR 条件不满足时冻结。真实复制 poll 在第 22 步后接入 report 与 HW 推进。

起点与编辑位置。

已给：ClusterAuthority 提供稳定的 per-TopicPartition lock/Condition、在线状态与进度容器；TimeSource 可控。

请修改：exercises/src/main/java/io/simplekafka/replication/ReplicationTracker.
java: report(int,int,long)、expireLagging()、advanceHighWatermark()。

行为契约与不变量。只接受 assigned/online/当前 epoch 且 LEO 非负的报告；非法报告不修改状态。follower LEO≤leaderLEO 且同 epoch 不倒退。minISR 不足时 HW 冻结；满足时 HW=max(旧 HW,min(ISR progress))。report、expire 引起可见条件变化时唤醒等待者。时间边界由注入 clock 定义，过期副本不包括 leader。

分级提示（不是答案）

一级 先列出每个 follower 报告需要满足的“不超过”和“不倒退”两项边界。

二级 分别计算 ISR 足够与不足的候选 HW，注意高水位不能逆向。

三级 对同一组进度做一次追平、一份过期和一次 minISR 下降，观察报告、ISR、HW 与等待者唤醒。

可观测例子与测试点。leader/follower LEO=(5,3,4)，minISR=2 时 HW=3；某 follower 报告 5 后 HW=5。只剩 leader 且 leader append 到 6 时，minISR=2 不满足，HW 留在 5。超过 lag timeout 的 follower 移出 ISR，leader 不被删除。对应 Step22Test 的可控时钟检查。

常见误区与真实 Kafka 的差异

把所有在线 replica 当 ISR 会把落后副本计入确认；用尚未成功 report 的 local LEO 计算 HW 会使读者看到未验证的数据。该课程 authority 保存已提交前缀，并在受控切换要求干净候选不低于已知 HW，这是额外教学假设；真实 Kafka HW 跨 epoch 不保证本课程所述的全局单调性。

命令与晋级。 `./gradlew :stepTest -Pstep=22` 用于定位本步；`./gradlew :test -Pstep=22` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么 minISR 不足时已追加数据也不能推进 HW?
2. follower 只是在线但未追到 leaderLEO，为何不能刷新 caught-up 时间?

7.3 第 23 步：生产确认

目标与前置知识

本步目标：把生产者的确认语义与复制状态、deadline 连接起来，区分“leader 已写”和“副本已提交”。

开始前应能解释：理解 Condition 等待、epoch 围栏和不可回滚 timeout。

背景与推理

ACKS 只有 LEADER=1 和 ALL=-1。LEADER 在 leader 日志追加完成后可返回，但切换时尾数据可能丢失。ALL 必须先验证 $ISR \geq \text{minISR}$ ，再追加并等待 HW 到达 `batch.nextOffset`；等待期间可能发生 minISR 下降或 leader epoch 变化。Condition 属于 authority partition lock，`awaitNanos` 会释放锁，让复制报告能进入；醒来后重新检查 epoch、ISR、HW 与 `deadline`。`deadline` 使用 `System.nanoTime` 的单调时钟，timeout 不代表追加回滚。所有 socket 网络调用都在 partition lock 之外。

起点与编辑位置。

已给：ReplicationTracker 绑定 authority condition 并提供 snapshot/锁；RecordData、AppendResult、Acks、TimeSource 已提供。

请修改：`exercises/src/main/java/io/simplekafka/replication/AckPolicy.java`：
`validateBeforeAppend(Acks)`、`await(AppendResult,Acks,int,long)`。

行为契约与不变量。 LEADER 追加完成即可返回。ALL 在追加前若 ISR 不足报 `NOT_ENOUGH_REPLICAS`；追加后等待 $HW \geq \text{nextOffset}$ 。等待中 epoch 变化报 `FENCED_EPOCH`，ISR 跌破 minISR 报 `NOT_ENOUGH_REPLICAS`，超 deadline 报 `REQUEST_TIMEOUT`。timeout 不回滚已写日志，deadline duration 非负。

分级提示（不是答案）

- 一级 区分追加前的可行性检查与追加后的等待条件，错误时点不相同。
- 二级 Condition 唤醒只是“状态可能变化”的通知，不代表条件已经满足。
- 三级 用 latch 控制副本报告顺序，让 ALL 请求先等待、再由 report 解锁；另行验证 timeout 后 LEO 与 HW 的差别。

可观测例子与测试点。ALL 追加后副本未报告时请求仍未完成；报告使 HW 到达 nextOffset 后成功。等待超时会留下已推进的 leader LEO，但消费者仍看不到 HW 之上的记录；LEADER 请求可以成功而记录尚不可见。对应 Step23Test 的线程协调测试。

常见误区与真实 Kafka 的差异

持锁等待复制网络会把唯一进度入口堵死并造成死锁；把超时请求重发会形成重复记录；用 wall clock 计算 deadline 会受到时间校准跳变。真实 Kafka 的 acks/ISR 配置和请求队列更复杂，课程仅支持两个 ACK 模式且不自动重试。

命令与晋级。./gradlew :stepTest -Pstep=23 用于定位本步；./gradlew :test -Pstep=23 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. awaitNanos 释放锁后为什么必须重查 epoch 和 ISR？
2. ALL timeout 后，LEO、HW 与调用方是否知晓追加之间是什么关系？

7.4 第 24 步：并发副本 broker

目标与前置知识

本步目标：在可并行服务请求的 broker 中组合角色检查、追加、复制确认与 HW 截断 fetch。开始前应能解释：理解锁顺序、跨 socket 等待和关闭时唤醒。

背景与推理

ReplicatedPartition 把 authority 状态和本地日志连接成业务操作。produce 在 authority partition lock 下核验 leader/epoch、追加、更新自身 progress 并推进 HW；ALL 等待时释放 partition lock，让另一 TCP worker 报告 follower 进度。fetch 只能读 HW 以下的完整前缀：合法范围中的 offset>=HW 返回空，超过 LEO 才是位点越界。网络 fetch replication 也必须在锁外执行，不能拿着 authority 锁等待另一 broker socket。锁顺序固定 authority lock→local log lock，不反向取锁、不嵌套不同 partition 锁。关闭 socket/调度器需唤醒 ACK waiter 并让它们以错误完成。

起点与编辑位置。

已给：复制 broker 的独立 endpoint、broker worker、ReplicaState、ReplicationTracker、Ack-Policy 与 PartitionLog 均已提供；单机 handler 继续复用。

请修改：exercises/src/main/java/io/simplekafka/replication/ReplicatedPartition.java：produce(List<RecordData>,Acks,int,long)、fetch(long,int,int,int)；BrokerHandler 显式路由 replicated backend。

行为契约与不变量。produce 按“角色与 epoch 验证 → 追加前 ACK 检查 → 日志追加 → leader report/HW 更新 → 确认等待”维持线性化；不能持有 partition lock 等网络。fetch 返回

offset<HW 的记录并裁剪总预算; offset 位于 [logStart,LEO] 且 $\geq$ HW 返回空, offset>LEO 越界。epoch 不匹配 fenced。单机与 replicated backend 显式选择, 不靠捕获异常回退。

分级提示 (不是答案)

- 一级 画出 produce 线程、follower poll 线程和 fetch 线程各自获取的锁, 标出网络边界。
- 二级 检查 fetch 的 HW 限制是在 broker 返回给普通消费者之前生效。
- 三级 暂停一台 follower 的复制, 保持另一条 socket 发起 fetch, 同时让 ALL produce 等待, 再恢复复制观察三个操作能继续。

可观测例子与测试点。暂停 follower 复制, ALL produce 等待期间另一连接 fetch 只能看到已提交前缀; 恢复 follower 并上报后 produce 成功, 新批次可见。关闭 broker 时 pending 请求以错误完成且 JVM 无遗留阻塞 worker。对应 Step24Test 的真实 TCP 并发验收。

常见误区与真实 Kafka 的差异

网络调用持锁可使复制线程永远拿不到锁, 单线程测试未必暴露; 把 HW 与 committed group offset 共用会混淆复制可见性与消费进度。真实 Kafka 运行多进程并包含复杂网络分区控制器行为; 这里的独立网络/磁盘仍共享 JVM 的单 authority, 不证明生产 HA。

命令与晋级。 ./gradlew :stepTest -Pstep=24 用于定位本步; ./gradlew :test -Pstep=24 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿, 才算完成本章并进入下一章。

自检问题

1. 为什么 ALL 等待期间另一条 fetch 可以安全并发读取旧 HW 前缀?
2. 关闭时只关 socket 是否足以保证等待 Condition 的线程结束?

第八章 epoch 围栏、干净选举、分叉修复与重新入组

本章将副本状态变化串成受控故障切换。这里必须牢记模型限制：本课程的 authority 是单 JVM 内可信的 metadata/HW 控制面，leader 切换由课程显式触发，不是 KRaft 控制器共识，也不覆盖网络分区下的双主仲裁。follower 通过真实 TCP 读取 leader 数据，每个 broker 仍写自己的独立日志。课程选择干净候选，不以数据丢失换取可用性；没有候选时服务必须保持不可用。

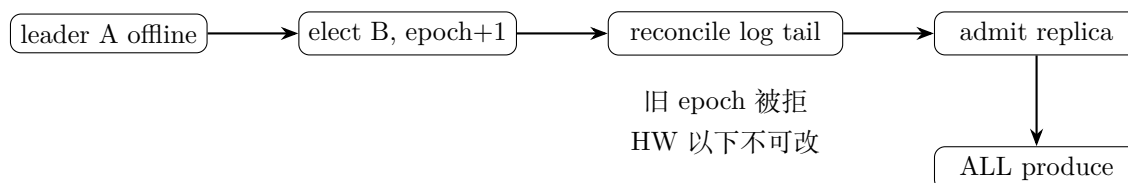


图 8.1: 角色切换必须先围栏旧角色，再修复并证明副本完整，最后才可重新进入 ISR。

8.1 第 25 步：干净选举与 epoch

目标与前置知识

本步目标：在旧 leader 离线时选择有已提交前缀的候选，并让旧 epoch 请求失效。

开始前应能解释：理解 ISR 候选集、HW 下界与 epoch 的角色围栏。

背景与推理

选举是角色决策而不是简单选择在线节点。当前 leader 仍在线时返回当前 metadata，不增加 epoch。否则候选只从旧 ISR 中选择 online 且 $LEO \geq \text{authority.HW}$ 的 broker，以最小 broker id 决定；低于已知提交边界的副本不可当干净候选。没有候选报 `NO_ELIGIBLE_LEADER`，不改 epoch/HW，也不进行 unclean election。成功角色变化使 epoch 加一，旧 leader 被 fence，新 leader 作为初始唯一 ISR，其他成员处于 recovering 状态；它们必须经过修复与 admission 才能重新参与确认。旧 epoch 的等待生产请求要被唤醒并失败。

起点与编辑位置。

已给：authority 持有可信 broker 在线状态、每分区锁、LEO/HW 与角色发布能力；三个 broker 的传输和日志不需重写。

请修改: `exercises/src/main/java/io/simplekafka/replication/LeaderElection.java`:
`elect(TopicPartition)`、`checkLeader(int, TopicPartition, int)`。

行为契约与不变量。当前 leader 在线时 metadata 和 epoch 原样返回。切换只从旧 ISR 选 online 且 $LEO \geq HW$ 的最小 broker id; 成功 epoch 恰加一、HW 不回退、新 ISR 初始只含 leader。无候选失败不变更状态。请求 epoch 旧报 `FENCED_EPOCH`; epoch 正确但 broker 非 leader 报 `NOT_LEADER`。

分级提示 (不是答案)

- 一级** 列出候选资格时先应用 ISR 限定, 再检查在线与 LEO/HW 。
- 二级** 把“epoch 正确”与“请求者是 leader”分开检查; 二者表示不同拒绝原因。
- 三级** 构造三个副本: 一个旧 leader offline、一个 ISR 候选低于 HW、一个 online 但不在 ISR, 验证只有符合全部条件者能当选。

可观测例子与测试点。A 是 epoch 0 leader, $HW=3$; A 停止后, 旧 ISR 中 $LEO \geq 3$ 的最小 broker B 当选并变 epoch 1, offset 0-2 可读。旧 epoch produce/fetch 被 fence; 若旧 ISR 全部离线, 即使 C 在线也不选且 $HW/epoch$ 不变。对应 `Step25Test`。

常见误区与真实 Kafka 的差异

仅选最先进的在线副本会把非 ISR 甚至未知分叉引入提交历史; 没有候选时偷偷选落后副本属于 unclean election。真实 Kafka 通过控制器、leader epoch 和集群状态处理选举; 本课程的强 authority 假设并不覆盖失去 authority 的恢复, 也不证明跨网络分区一致。

命令与晋级。`./gradlew :stepTest -Pstep=25` 用于定位本步; `./gradlew :test -Pstep=25` 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿, 才算完成本章并进入下一章。

自检问题

1. 候选 $LEO \geq HW$ 保证了什么最小安全条件?
2. 为什么 epoch 正确的非 leader 仍然不能接受 produce?

8.2 第 26 步: 分叉日志修复

目标与前置知识

本步目标: 让旧副本与新 leader 找共同前缀、截断未提交分歧并追上 leader。

开始前应能解释: 理解 HW 保护、恢复读取、epoch 变化和证明的作用范围。

背景与推理

课程修复采用全前缀扫描这一教学简化: 从 offset 0 到捕获的 leader LEO 分批使用 `recoveryRead=true` 标志的 `REPLICA_FETCH`, 比较每个本地已有记录与 leader 对应记录。恢

复读取能看到 HW 之上的 leader 尾部，但不报告复制进度、不修改 ISR/HW。本地已有且 offset < captured HW 的记录若不同，报 CORRUPT_RECORD 且不改文件；本地只是缺记录则可补齐。扫描前捕获当前 PartitionLog 对象的 mutationVersion，读取完整本地前缀后，在 authority 锁内重核 epoch、start、LEO 与 version，并在同一锁序下持有本地日志 monitor 直至修复及 proof 发布；若扫描期间经日志 API 发生任何修改（包括截断后重新追加到同一 LEO），报 REQUEST_TIMEOUT，不得用变化后的日志做判断/截断或发布 proof。只有检查稳定后才允许从 HW 起修复分歧并追加 leader 内容；全程网络读取在 authority 锁外。成功 proof 绑定 PartitionLog 对象身份及最终 mutationVersion，以免 close/reopen 后版本重置误用旧证据；该版本仅是单个打开对象的内存代数，不是持久校验，也不识别绕过 API 的直接磁盘篡改。全扫描开销 O(n)，仅用于显式恢复。

起点与编辑位置。

已给：PartitionLog 的 truncateTo/append、RpcClient 的 replica fetch、authority 的 epoch/HW 快照和分区锁已提供；没有自动把旧副本当作 ISR。

请修改：exercises/src/main/java/io/simplekafka/replication/ReplicaReconciler.java: reconcile(intexpectedEpoch)。

行为契约与不变量。预期 epoch 已非当前时以 FENCED_EPOCH 退出。本地已有且 offset 小于 captured HW 的记录若与 leader 不同，报损坏且不改文件；缺失提交前缀可以修复。扫描期间本地 mutationVersion 改变时以 REQUEST_TIMEOUT 退出且不修日志。最终重核、修复与 proof 发布须在 authority 分区锁及本地日志 monitor 的既定锁序下原子进行；任何截断点不得小于 HW。结束时返回 LEO，并生成同时绑定捕获 epoch、leader LEO、本地日志对象身份与最终 version 的 proof；admission 必须重新核验 proof。恢复期间不报告 ISR progress。

分级提示（不是答案）

- 一级 比较数据时要把“本地缺记录”与“本地有不同记录”分开。
- 二级 识别哪些位置由 captured HW 保护，哪些未提交后缀可改变。
- 三级 构造已提交共同前缀、分叉未提交尾部和落后空副本三种情形，并在修复后逐 offset 验证磁盘内容。

可观测例子与测试点。已提交 0-2、A 有 X@3；切到 B 后 B 写 Y@3，A reconciliation 后 offset 0-2 不变、offset 3 为 Y、LEO=4。空副本也能复制完整提交前缀。若 offset 1 在两端不同且 captured HW=3，则拒绝并保持 A 文件不变。对应 Step26Test。

常见误区与真实 Kafka 的差异

将本地 HW=0 机械当作“可以删掉所有数据”会丢失实际已复制的提交前缀；把缺失前缀当成分叉损坏会阻止正常追赶。真实 Kafka 通过 leader epoch 历史定位分歧并精确截断，KIP-101 解释为何不能只用 high watermark 推断截断点；本课程 O(n) 扫描便于教学，不代表生产做法。

命令与晋级。 ./gradlew :stepTest -Pstep=26 用于定位本步；./gradlew :test -Pstep=26 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为何“已提交记录不同”和“本地缺少已提交记录”应采用不同处理？
2. leader 在捕获快照后又写入，旧 proof 为何不能直接用于 admission？

8.3 第 27 步：元数据路由刷新

目标与前置知识

本步目标：让客户端缓存 metadata 并按分区 leader 路由，同时避免错误后自动重发业务请求。

开始前应能解释：理解控制面查询、数据面路由和不确定确认。

背景与推理

bootstrap endpoint 只是发现入口，不必永久承载数据。refresh 按 bootstrap 顺序查 metadata，某个连接失败可继续下一个可用节点。call 按缓存 leader/epoch 发送一次业务请求。收到 NOT_LEADER 或 FENCED_EPOCH 时可刷新缓存以便下一次请求使用新角色，但必须把原请求的原始错误传播给调用者，不能使用新 metadata 自动重放业务请求：旧请求的错误也可能发生在等待确认期间，数据可能已追加。I/O、timeout、storage 错误同样不自动重发。producer、consumer、group consumer 可增加 Router 构造入口；原 RpcClient 构造器继续支撑单机模式，不作为待废弃 shim。group/offset 控制面使用可用 bootstrap control client，partition fetch/produce 走 leader。

起点与编辑位置。

已给：RpcClientFactory 能按 Endpoint 建连接；MessageCodec 与原客户端逻辑已完成；broker metadata 返回真实 leader endpoint/epoch。

请修改：exercises/src/main/java/io/simplekafka/client/MetadataRouter.java：refresh(String)、call(TopicPartition, Request)；接入 SimpleProducer、SimpleConsumer、GroupConsumer 的 Router 构造路径。

行为契约与不变量。refresh 只读 metadata 并按 bootstrap 顺序容错；call 使用缓存 endpoint 和 epoch。NOT_LEADER/FENCED_EPOCH 可触发 refresh，但必须传播最初响应，不得重发 produce/fetch/commit。其他网络与存储错误不自动重试。后续新的独立请求才使用新缓存。

分级提示（不是答案）

- 一级 明确一个 call 有且只有一次业务 request attempt，刷新是控制面动作，不是重试。
- 二级 不同 endpoint 失败顺序对初始 metadata 查询有什么影响？
- 三级 模拟旧缓存写入被拒但服务端可能已追加，检查刷新后旧 receipt 与记录数，再发下一次显式请求。

可观测例子与测试点。旧 metadata 指向 A/epoch 0；A 停止后第一次路由请求收到刷新相关错误，客户端刷新到 B/epoch 1 但把该错误返回。显式第二次新请求只向 B 写入一份。若 broker 已追加但响应丢失，客户端报错且不会自动生成第二条。对应 Step27Test。

常见误区与真实 Kafka 的差异

把 NOT_LEADER 当作“肯定没有写入”会造成 duplicate produce；只刷新缓存却向调用方返回成功又会伪造 receipt。MetadataRouter 会刷新但不会重放请求，SimpleProducer 也不会自动重试。第 8 章另设 IdempotentProducer，由调用方显式 retry 并沿用持久化 batch identity；它不会改变普通 producer 契约，也不提供事务或端到端 exactly-once 保证。

命令与晋级。 ./gradlew :stepTest -Pstep=27 用于定位本步；./gradlew :test -Pstep=27 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么刷新 metadata 与重放业务请求是两个不同的动作？
2. 客户端看到错误时，为什么不能断定服务端没有产生副作用？

8.4 第 28 步：副本重新入 ISR

目标与前置知识

本步目标：把已修复副本经过严格证明后重新纳入 ISR，并验证综合故障切换。

开始前应能解释：理解副本的分配、在线状态、follower 角色、epoch 与 proof 的有效期。

背景与推理

副本恢复完成不自动等于同步。日志修复生成的前缀 proof 才能申请重新入 ISR，并绑定捕获 epoch、leader LEO、本地 PartitionLog 对象身份与 mutationVersion。authority 分区锁下检查目标 broker 已分配且在线、处于当前 follower 角色、expected epoch 与当前一致；持有目标 PartitionLog 的 monitor 直到 ISR 转换发布，append/delete/truncate/close 也使用同一 monitor，因此证明核验与入 ISR 之间没有本地日志变更窗口。proof 仍须绑定当前 leader epoch/LEO 和当前传入的同一个日志对象/version，日志 start 必须为 0，当前 LEO 必须精确等于 leader LEO。任何经 PartitionLog API 的 append/delete/truncate 都使旧 proof 失效，即使 truncate 后追加恢复相同 LEO；需要重新 reconciliation。这个内存版本不能发现绕过 API 的外部文件修改。成功入 ISR 时初始化 reportedLEO/lastCaughtUp 并重算 HW；重复添加已在 ISR 的成员返回 true 但不改变 HW。

起点与编辑位置。

已给：authority 的角色/ISR 管理、复制进度、proof 类型与 broker 生命周期已由课程基础层提供。

请修改：exercises/src/main/java/io/simplekafka/replication/ReplicaAdmission.java

```
va
: tryAdd(int, TopicPartition, int)。
```

行为契约与不变量。目标副本已分配、在线、是当前 epoch 的 follower，且 proof 的 leader 与本地日志对象身份/version 正确；日志 start=0、本地 LEO= 当前 leader LEO，才返回 true。proof 核验期间本地 PartitionLog API 写操作不能穿插，leader 追加或任何本地 PartitionLog 变更后旧 proof 返回 false；unassigned 报 INVALID_REQUEST，offline 或 proof 不新鲜返回 false。已在 ISR 重复调用 true，不改变高水位。成功后重算 HW 并唤醒确认等待者。

分级提示（不是答案）

- 一级** 将 proof 的有效性看作版本化的事实，不是永久布尔标志。
- 二级** leader LEO 在 proof 产生后变化时，需要重新比较哪些状态？
- 三级** 先在简化用例验证无变化时可入组，再让 leader 追加一条后重用旧 proof，最后以新 proof 重试。

可观测例子与测试点。停止 A，B 接管；修复并 admission 存活 C 后恢复 minISR；ALL 写入新消息。重开 A，修复并 admission，再次 ALL 写入；三份独立日志记录字节/offset 一致且无间隙，group 从 committed offset 继续。重用旧 proof 在 leader LEO 改变后必须 false。对应 Step28Test 的完整 TCP 集成验收。

常见误区与真实 Kafka 的差异

只检查 LEO 相等而不比较前缀，会让分叉副本冒充同步；只核对 proof 生成时 epoch 而不核对当前 leader LEO，会把新尾数据漏掉。真实 Kafka 的 ISR 进入与 leader epoch/复制协议更复杂；课程严格的前缀 proof 和强 authority 是可验证教学约束，不等于生产控制器设计。

命令与晋级。 ./gradlew :stepTest -Pstep=28 用于定位本步；./gradlew :test -Pstep=28 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. LEO 一致为什么不足以证明两份日志的内容一致？
2. 为什么 admission 成功后还要重算 HW 并唤醒等待确认的请求？

第九章 生产者、broker 与消费者的一致性

本章沿着同一消息观察三个彼此独立的边界：producer 确认与重试、broker 复制与可见性、consumer 进度与持久化业务副作用。某一边界成功不构成端到端事务。实验使用真实 TCP 连接和每个 broker 独立的日志；观察结果分别记录物理日志末端（LEO）、高水位（HW）、consumer position、已提交 offset 与 processor 回调。

9.1 第 29 步：三层一致性故障实验

目标与前置知识

本步目标：编排可复现的 producer、broker、consumer 故障，并逐层报告实际发生的状态。

开始前应能解释：理解成功追加、响应丢失、HW 可见性、consumer 内存 position、group committed offset 与外部副作用的区别。

背景与推理

已提供的 `FaultProxy` 会把原始 frame 转发给真实 broker，可选择转发匹配请求后丢弃回复，或在转发前拒绝匹配请求。`EffectRecorder` 记录每次回调，包括重复回调。`ConsistencyObservation` 捕获阶段名、错误码、复制快照、所有 broker 的物理 LEO、从当前 leader 的 HW 以下真实读取的记录、可选 consumer position 和 group committed offset、已观察副作用及可选追加 receipt。`ConsistencyTrace` 把同一分区的这些观察组成轨迹。每个场景均从调用方已启动的 `ClusterHarness` 开始：一个空单分区 topic、leader=1、三个副本在 ISR 中、minISR=2、HW=0、三份物理 LEO 均为 0、没有已提交 group offset，输入记录没有 stamp。需要故障注入的场景还接收一个已启动、尚未 arm、upstream 为 broker 1 的 proxy。使用注入的固定 clock 与已提供的复制 helper，不通过 sleep 或 expiry 竞态制造结果。其 API 包括 `FaultProxy(Endpoint)`、`upstream()`、`start()`、`endpoint()`、`armed()`、`dropNextReply(short)`、`rejectNextRequest(short)`、`forwarded(short)`、`dropped(short)`、`rejected(short)` 与 `close()`。同一时刻只能 arm 一个 API 故障。drop fault 会先转发请求并读取真实 broker 回复，再关闭下游连接而不回回复；reject fault 不转发请求，返回 `REQUEST_TIMEOUT` 和 `ErrorBody("injectedpre-forwardfailure")`。`LabSupport` 提供 `replicateAndReport(cluster, tp)`、`observe(phase, error, cluster, tp, position, group, effects, receipt)` 与 `awaitLeaderLeo(cluster, tp, expectedLeo)`。观察字段按序为 phase, error, replication, physicalLeo, position, committedOffset, visible, effects, receipt；visible 来自真实 leader 在 HW 以下的 FETCH，physical LEO 来自每份日志

而非 authority 报告。EffectRecorder 实现 RecordProcessor；同步的 process(tp,record) 追加每次回调，同步的 snapshot() 返回稳定副本而不去重。ConsistencyObservation 的 record 签名为 recordConsistencyObservation(Stringphase,ErrorCodeerror,ReplicationSnapshotreplication,Map<Integer,Long>physicalLeo,OptionalLongposition,OptionalLongcommittedOffset,List<LogRecord>visible,List<LogRecord>effects,Optional<AppendResult>receipt)；ConsistencyTrace 为 recordConsistencyTrace(TopicPartitiontp,List<ConsistencyObservation>observations)。

起点与编辑位置。

已给：FaultProxy、EffectRecorder、ConsistencyObservation、ConsistencyTrace 和 LabSupport 提供故障注入、效果记录和真实状态观察。

请修改：exercises/src/main/java/io/simplekafka/lab/ConsistencyExperiments.java：producerOutcomeUnknown(ClusterHarness,TopicPartition,RecordData,FaultProxy)、leaderAckLoss(ClusterHarness,TopicPartition,RecordData)、allAckTimeout(ClusterHarness,TopicPartition,RecordData)、consumerReplay(ClusterHarness,TopicPartition,String,RecordData,EffectRecorder,FaultProxy) 与 staleGroupSideEffect(ClusterHarness,TopicPartition,String,RecordData,EffectRecorder)。

不匹配的 API 原样透传；proxy 计数仅在实际转发、丢弃回复或转发前拒绝时增加。API 无效或尝试同时 arm 第二个故障时报告 INVALID_REQUEST。五个可编辑方法均为 public static，返回 ConsistencyTrace；consumerReplay 还声明 throwsException。

行为契约与不变量。使用调用方提供的数据实现全部五个方法。初始条件为已启动 cluster、一个空 partition、leader 1、ISR=1,2,3、minISR=2、HW=0、所有物理 LEO=0、输入未带 stamp；consumer 场景还要求 group 尚未提交且调用方持有的 recorder 为空。两个故障注入方法要求传入已启动、upstream 为 broker 1 且未 arm 的 proxy。初始状态无效时报 INVALID_REQUEST。只关闭场景自有 client；仅记录预期故障，其余失败必须传播。producerOutcomeUnknown 丢弃一次成功 LEADER 回复并报告 UNKNOWN (LEO 1/0/0、HW=0、position=0、没有可见记录和 receipt)；再次使用该 SimpleProducer 时报告 POISONED，调用被拒且没有额外转发 PRODUCE。独立的普通 producer 显式重发相同数据并取得 [1,2)；复制后报告 EXPLICIT_RESEND (LEO 2/2/2、HW=2、position=2、offset 0 与 1 的可见记录内容相同，且不观察 group committed offset)。leaderAckLoss 获得 LEADER receipt [0,1) 但不复制，报告 ACKED_NOT_VISIBLE (LEO 1/0/0、HW=0、position=0)；停止 broker 1，选举 broker 2、epoch 1，报告 ELECTED_WITHOUT_TAIL；重启 broker 1 并 reconcile，报告 OLD_TAIL_REMOVED (所有 LEO=0、HW=0、无可见记录)。allAckTimeout 使用 ALL、timeout=0，报告 TIMEOUT_NOT_VISIBLE (LEO 1/0/0、HW=0、position=0、无 receipt)；真实复制并 report 后，同一 consumer 读取 offset 0，报告 LATE_VISIBLE (LEO 1/1/1、HW=1、position=1)，不重发，也不追溯性地补成成功 ACK。consumerReplay 写入一条并复制到 HW=1；consumer 读取 offset 0 后，proxy 在转发前拒绝 COMMIT_OFFSET。报告 COMMIT_FAILED，错误为 REQUEST_TIMEOUT、position=1、无 committed offset、effects=[0]。关闭该 consumer；新连接从 0 恢复，再次处理并提交，报告 REPLAY_COMMITTED，position 和 committed offset 均为 1，effects=[0,0]。staleGroupSideEffect 让 member z 加入、读取 offset 0 并记录副作用，再让 member a 加入并取得分配。z 的旧 token 对 COMMIT 和 FETCH 均收到 ILLEGAL_GENERATION；报告 STALE_COMMIT_REJECTED (position=1、无 committed offset、effects=[0])。当前 owner 读取

offset 0、记录副作用并提交 nextOffset=1；报告 NEW_OWNER_COMMITTED (committed offset=1、effects=[0,0])。fencing 会拒绝旧请求，但无法撤销先前回调。

UNKNOWN 与 POISONED 两个观察阶段的错误均为 REQUEST_TIMEOUT；poisoned reuse 在本地被拒绝，不增加 proxy 转发数。

分级提示（不是答案）

- 一级 分别追踪各 broker 的物理 LEO，不要把它与 authority 报告的 LEO 或 HW 混为一谈。
- 二级 为每个故障写时间线，并标出错误发生时哪一方已经完成了动作。
- 三级 测试由调用方持有 recorder 和 proxy：除返回轨迹外，也要独立断言它们的计数与快照。

可观测例子与测试点。逐阶段对照真实 cluster 及调用方持有的 recorder/proxy。响应丢失时断言 dropped(PRODUCE)=1、forwarded(PRODUCE)=2：poisoned attempt 不会转发，只有新 producer 的显式重发贡献第二次转发。选举轨迹中的 ELECTED_WITHOUT_TAIL 仍为 LEO 1/0/0、HW=0，随后 OLD_TAIL_REMOVED 为 LEO 0/0/0。ALL 故障为 REQUEST_TIMEOUT；稍后可见不会追溯性地产生 receipt。consumer replay 时断言 rejected(COMMIT_OFFSET)=1、forwarded(COMMIT_OFFSET)=0、effects=[0,0]，新 consumer commit 后真实 FETCH_OFFSET 返回 1。stale group replay 时旧 token 的两次请求都被 fencing，但 effects 仍为 [0,0]。

常见误区与真实 Kafka 的差异

receipt 证明的是 ACK 结果，不保证 LEADER 已确认但未复制的尾部会在之后的干净选举中保留。timeout 不能证明没有追加。poll 推进 consumer position 早于 group commit；group fencing 保护 broker 操作，不会撤销副作用。这些实验不会给 SimpleProducer 增加重试，不用 sleep 制造 timeout，也不声称端到端 exactly-once。

命令与晋级。 ./gradlew :stepTest -Pstep=29 用于定位本步；./gradlew :test -Pstep=29 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 哪些观察能区分 LEADER 确认与复制提交后的可见性？
2. 为什么拒绝 stale commit 的同时，先前的副作用仍然可以存在？

9.2 第 30 步：幂等生产者

目标与前置知识

本步目标：让每条记录携带可持久化的 producer batch 身份，并使显式重试返回原 offset 区间而不重复追加。

开始前应能解释：理解 batch sequence、producer epoch、内容指纹、部分复制，以及为什么重试必须是调用方显式选择。

背景与推理

原有三参数 `RecordData` 构造器仍是普通、非幂等记录的正式入口。新增的 nullable `producerStamp` component 是 broker 为幂等请求设置的 metadata。`ProducerStamp` 含 `producerId`、由调用方管理的 `producerEpoch`、`firstSequence`、`batchSize`、`batchIndex` 与 32-byte `batchHash`。SHA-256 指纹以流式方式计算 `count` 以及每条记录的 `timestamp`、`key` 长度/`key`、`value` 长度/`value`；不包含 `offset` 或 `leader epoch`，并区分 `null key` 与空 `key`。该指纹用于验证相同身份的重试是否携带同一 `batch`。合法 `stamp` 要求 `producerId`、`producerEpoch`、`firstSequence` 非负；`batchSize` 为正；`batchIndex` 在 `[0,batchSize)`；`firstSequence+batchSize` 不溢出；`hash` 恰好 32 bytes。指纹输入为大端 `count:int32`，之后逐条输入 `timestamp:int64|keyLength:int32(-1meansnull)|key|valueLength:int32|value`；不包含 `offset` 或 `leader epoch`。

已提供的 `ProducerStamp.ENCODED_OVERHEAD` 为 64 bytes；磁盘与 `MessageCodec` wire 的 1 MiB record 上限均计入这部分开销。每条 stamped record 都在普通日志记录中保留身份，因此 `FETCH`、`REPLICA_FETCH`、`reconciliation` 与磁盘恢复会看到相同 metadata。stamped 磁盘布局为 `length|crc32c|offset|timestamp|marker:int32=-2|producerId:int64|producerEpoch:int32|firstSequence:int64|batchSize:int32|batchIndex:int32|batchHash:32bytes|keyLength:int32|valueLength:int32|key|value`。它额外占 64 bytes，最小记录总长为 96 bytes；1 MiB 记录上限包含 `stamp`，`CRC32C` 覆盖 `offset` 到 `value` 末尾。普通未 stamped 字节不变。教学 wire 上的 stamped `RecordData` 为 `marker=-2|producerId|producerEpoch|firstSequence|batchSize|batchIndex|batchHash[32]|keyLength|key|valueLength|value|timestamp`；普通 wire record 不变。

broker 从保留日志前缀派生去重状态，而不是将它作为新的 `authority` 真相。它在写入前验证 `sequence` 与完整 `batch hash`，记住已完成 `batch` 的 `offset` 范围，并允许复制 `fetch` 追加匹配的部分 `batch` 后缀。完整重复返回原范围；补齐部分尾 `batch` 的重试只追加缺失后缀。更高 `producer epoch` 必须从 `sequence 0` 开始，且只有实际持久化记录后才生效。同一 `batch` 身份但 `hash` 不同会冲突；`sequence` 跳号、未记录的旧 `sequence` 或非法边界属于顺序错误。未完成尾 `batch` 会阻止无关追加，直到原 `batch` 补齐或正常修复改写前缀。若保留日志起点大于 0，则 `producer state` 过期，不能猜测恢复。

起点与编辑位置。

已给：`stamp` 值、指纹 helper、记录/请求序列化支持和复制日志基础设施已提供。需扩展既有存储与路由算法，并实现新 client 与复制写入路径。

请修改：`exercises/src/main/java/io/simplekafka/client/IdempotentProducer.java`: `send(TopicPartition,List,Acks,long)`、`retry(TopicPartition)`；`exercises/src/main/java/io/simplekafka/replication/IdempotentAppender.java`: `append(long,int,long,List)`、`recover()`；`exercises/src/main/java/io/simplekafka/replication/ReplicatedPartition.java`: `produceIdempotent(List,Acks,int,long,long,int,long)`；`exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java`: `handle(IDEMPOTENT_PRODUCE)`；`exercises/src/main/java/io/simplekafka/client/MetadataRouter.java`: `call`；扩展 `RecordCodec.encode/decode`、`SegmentLog.append/recover` 与 `PartitionLog.append/deleteBefore/truncateTo`。

已提供的 `partition-log` 支持还包括 `tailProducerStamp()`、`prefixVersion()` 和 `truncateToOffset(long)`，以便追加校验与 `cache` 失效依据真实持久化尾部和被改写的前缀。已提供的 `IdempotentProducer(MetadataRouter, long, int)` 构造器与 `close()`，以及 `IdempotentAppender(PartitionLog)` 构造器，负责生命周期初始化；只需实现前述算法方法。`IdempotentAppender` 构造时不会自动调用 `recover`；恢复是显式算法调用。公共算法签名为：`send(TopicPartition, List<RecordData>, Ack, long)` 与 `retry(TopicPartition)` 返回 `ProduceReceipt`；`IdempotentAppender.append(long, int, long, List<RecordData>)` 和 `ReplicatedPartition.produceIdempotent(List<RecordData>, Ack, int, long, long, int, long)` 返回 `AppendResult`；`recover()` 返回 `void`。已提供的 `IdempotentProduceBackend` 能力由 `ReplicatedPartition` 实现；若 `backend` 不支持该能力（包括单 `broker backend`），`BrokerHandler` 对 API 11 报 `INVALID_REQUEST`。

行为契约与不变量。`IdempotentProducer.send` 在发出一次 RPC 前固定每分区 `sequence` 与完整输入 `batch` 的稳定副本。只有合法成功 `receipt` 才推进 `sequence` 并清除 `pending`。任何请求/回复失败都保留该 `batch`；同分区的另一 `send` 必须拒绝，而 `retry` 显式刷新 `metadata`，并使用相同身份、`sequence`、`payload`、`ACK` 模式和 `timeout` 发出且只发出一次请求。没有后台或自动重试；`close` 只丢弃本地 `pending`，不关闭借用的 `router`。`producer id` 与 `epoch` 由调用方提供；重建 `client` 必须使用更高 `epoch` 和 `sequence 0`。`IdempotentAppender` 在任何 `mutation` 前验证所有输入；精确完成批次的重复返回原范围，相同部分批次只追加缺失后缀，同身份但 `hash` 不同则报 `DUPLICATE_SEQUENCE_CONFLICT`。旧 `producer epoch` 报 `FENCED_PRODUCER_EPOCH`；跳号和非法旧 `sequence` 报 `OUT_OF_ORDER_SEQUENCE`；保留前缀缺失报 `PRODUCER_STATE_EXPIRED`；未完成尾 `batch` 时写入无关记录报 `INCOMPLETE_BATCH`。持久 `mutation` 失败或留下不确定状态后，该 `log` 标记 `failed`，在重开前不得继续操作。前缀截断或 `retention` 会使派生 `cache` 失效；恢复时重新扫描并验证保留历史。`wire API id` 为 11，即 `IDEMPOTENT_PRODUCE`，请求字段为 `tp, epoch, acks, timeoutMillis, producerId, producerEpoch, firstSequence, records`；成功响应复用既有 `ProduceBody`。空、超大、非法或已 `stamped` 的 `client batch` 必须无副作用地拒绝。此 API 只由 `replicated backend` 支持，单 `broker backend` 不支持。重复请求仍需通过 `ACK precheck` 并等待完整 `ACK` 条件，包括 `ALL` 的 `minISR`、`epoch`、`HW` 与 `deadline` 规则。

日志派生状态按 `producer id` 记录最高的已持久化 `epoch`、`next sequence`、已完成 `batch` 的 `identity/范围`；整个 `partition` 至多有一个未完成尾 `batch`。未知 `producer` 或更高 `epoch` 必须从 `sequence 0` 开始；当前 `epoch` 的新 `batch` 必须恰好从已保存的 `next sequence` 开始。恢复用有界批次读取并校验 `batch` 连续性与 `hash`；`prefix version` 不变时只重放新后缀。前缀改变或 `LEO` 回退时丢弃 `cache` 并从 `offset 0` 重建；恢复报错会使派生状态失效，不能跳过损坏历史。`client` 细节：`pending batch` 只阻止同一 `partition` 的另一 `send`，其他 `partition` 相互独立。成功 `receipt` 必须恰好覆盖该 `batch` 的记录数，才能推进 `sequence`。`retry` 在没有 `pending batch` 时报告 `INVALID_REQUEST`；`metadata` 刷新失败和非法回复均保留 `pending state`，`retry` 使用刷新后的当前 `leader epoch`。重复 `batch` 仍执行 `ACK precheck` 与完整 `ACK wait`。

分级提示（不是答案）

一级 区分三种身份：`leader epoch` 围栏 `broker` 角色，`producer epoch` 围栏 `producer` 会话，`sequence` 标识该 `producer` 会话内的 `batch`。

二级 把日志视作持久真相，把内存 producer-state cache 视作带版本的重放结果。

三级 测试一个完整重复、同 sequence 不同内容冲突、一个部分 batch 续传，以及 leader 切换后的显式重试；同时检查 receipt 区间与 stamped 磁盘记录。

可观测例子与测试点。发送多条 batch 后以相同身份和 payload 重试：两个 receipt 指向相同 half-open 区间，重复时物理 LEO 和 stamped bytes 均不变；下一 sequence 则从后续 offset 追加。首个回复丢失后，将记录复制至 HW、选举干净的新 leader，再显式重试，取得原范围且每个 broker 仍只有一份。将某 broker 的日志从磁盘重开后，以相同 raw 请求再发送，仍返回原范围。重复 ALL 请求在范围高于 HW 时仍会 timeout，只有复制并确认该范围后才能成功。

常见误区与真实 Kafka 的差异

这是调用方提供的教学身份；它没有实现 Kafka PID 分配、transaction coordinator 或 client session WAL。新的 producer id 不会去重重复业务事件，业务 event id 也不是 producer sequence。SimpleProducer 仍是幂等且从不重试；只有调用方调用 retry，新 IdempotentProducer 才重试。幂等追加不会强化 ISR、HW 或选举假设，也不会形成端到端 exactly-once 保证。

命令与晋级。 ./gradlew :stepTest -Pstep=30 用于定位本步； ./gradlew :test -Pstep=30 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么重复的 ALL 请求仍需检查当前 HW 与 minISR？
2. 无法从保留日志前缀重建 producer 去重状态时，会丢失什么信息？

9.3 第 31 步：消费副作用幂等

目标与前置知识

本步目标：即使 Kafka record 被重复投递，也只把每个稳定业务事件应用到账本一次。

开始前应能解释：理解投递去重与业务事件去重使用不同身份，以及外部副作用为何不与 consumer offset 原子化。

背景与推理

BusinessEvent 含稳定的 eventId、account 和带符号的 delta。两个标识均为非空、严格 UTF-8 且最长 255 bytes；delta 可正、可负或为零。其带版本 value 编码为 version:uint8=1|eventId:identifier|account:identifier|delta:int64。已提供的 MessageCodec.encodeBusinessEvent/decodeBusinessEvent 复用课程 identifier 编码，并拒绝非法或有尾随字节的输入。store 将编码后的 event id、account 与 delta 作为一条账本记录追加；内存身份表和余额表通过重放重建，而非分别持久化。codec 签名为 public static byte[] encodeBusinessEvent(BusinessEvent) 和

`public static BusinessEvent decodeBusinessEvent(byte[]);` 未知 version、短输入、非法 UTF-8 或标识、尾随字节均拒绝。消费 demo 的 processor 会用 `MessageCodec.decodeBusinessEvent` 解码 `record.data().value()`，再调用 `DurableEffectStore.apply`；既有 `ProcessingLoop` 仍保持先处理、后 commit 的顺序。已提供的 `balance(account)` 使用相同规则验证 account 标识；账户不存在时返回零。

起点与编辑位置。

已给：BusinessEvent、事件序列化，以及 `DurableEffectStore(Path)`、`balance(String)` 和 `close()` 已提供。

请修改：exercises/src/main/java/io/simplekafka/client/DurableEffectStore.java：实现 `apply(BusinessEvent)` 与 `recover()`。

已提供的 store API 为 `DurableEffectStore(Path)`、`synchronizedbooleanapply(BusinessEvent)`、`synchronizedlongbalance(String)`、`synchronizedvoidrecover()` 与 `synchronizedvoidclose()`；close 可重复调用。已提供的构造器会打开本地日志，并在 store 可用前通过 `recover()` 重建状态。

行为契约与不变量。 将 <directory> 用作本地 PartitionLog。每个事件只生成一条 `Record Data(null, encodedEvent, 0)`，先 force 追加记录，再发布内存状态。首次遇到合法 event id 时，先用 checked addition 计算账户新余额，再追加一次，随后把完整事件写入 seen 并更新余额；返回 true。event id、account、delta 完全相同的重放返回 false，不改日志或余额。同 event id 不同内容或余额溢出时报告 INVALID_REQUEST，且不追加。recover 按保留 offset 顺序重建两个 map，并要求 logStart=0、key=null、timestamp=0、没有 producer stamp。它解码每个 payload、验证 event identity，相同重复事件恢复时只计一次；完整非法记录、冲突 id、余额溢出或扫描无进度报告 CORRUPT_RECORD。apply、balance、recover、close 在同一 store 实例 monitor 下同步。追加失败且持久化结果未知时，不发布新内存余额并将 store 标记为 failed；后续操作报告 STORAGE_ERROR，调用方必须 close 并从磁盘重建。

恢复以有界读取按 offset 顺序扫描保留账本；物理尾部不完整时遵循底层日志的恢复规则，完整但格式非法的记录则拒绝。store 已关闭或失败后，apply、balance 与 recover 均报告 STORAGE_ERROR；close 可幂等重复调用。

分级提示（不是答案）

- 一级 event id 必须来自业务事件，并在不同 Kafka offset 与进程重启后保持不变。
- 二级 写日志前先校验 event id 冲突和 checked balance；只有追加成功后才发布内存状态。
- 三级 consumer 关闭并重开后，从新的消费位置重放同一事件，再同时比较余额与账本记录。

可观测例子与测试点。 处理事件 A (account +10)、第二条携带相同 A 的 Kafka record，以及事件 B (account -3)。第一个 A 与 B 各追加一次；重复 A 返回 false，余额为 7，账本事件为 [A,B]。若首轮 commit 失败，重建 consumer 并重放 offset 0、1、2；验证 callback history 为 [0,1,2,0,1,2]、账本仍是两条事件、余额为 7，committed offset 为 3。stale generation 可以拒绝 commit，却不能撤销已持久化副作用。

常见误区与真实 *Kafka* 的差异

该 store 只让单个本地账本追加与内存发布保持一致；它与 *Kafka* 或 consumer offset store 没有共享事务。同一事件可以触发多次 callback，包括出现在不同 offset 的记录。group fencing 会拒绝旧 token，但不能回滚副作用。这是单个本地 store 的应用层幂等记账，不是分布式事务、全局账本或端到端 exactly-once 保证。

命令与晋级。 `./gradlew :stepTest -Pstep=31` 用于定位本步；`./gradlew :test -Pstep=31` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么稳定 event id 比 *Kafka* offset 或 callback 次数更适合作为去重键？
2. 账本追加成功但 consumer offset commit 失败时，哪些状态仍未协调？

附录 A 格式、错误码与接口索引

本附录把各章反复使用的稳定约定集中列出，方便实现时查阅；若和练习里的公开签名不一致，以课程 manifest 和 Java 类型为准。本课程定义的布局与错误码只属于 simpleKafka 教学接口，除非明确引用官方资料，不应把它们称为 Kafka wire、存储格式或配置默认值。

A.1 磁盘与网络格式

A.1.1 记录与索引

记录为大端编码：

字段	宽度	语义
length	int32	不含自身的记录长度，含 CRC；28 至 1,048,576。
crc32c	int32	校验 offset 到 value 末尾，不含 length/CRC 字段自身。
offset	int64	非负逻辑位点。
timestamp	int64	非负时间戳。
keyLength	int32	-1 为 null，0 为存在的空 key，其余为 bytes 数。
valueLength	int32	非负 bytes 数，value 不为 null。
key/value	变长	按前述长度紧跟在字段后。

每条记录的固定长度部分（不含 length 前缀）包含 CRC、offset、timestamp、两个 length，共 28 bytes；长度前缀使最小总占用为 32 bytes。length 字段值下限仍是 28，因为其统计从 CRC 开始。每条 segment 的稀疏索引条目是 `offset:int64,position:int64`，共 16 bytes；索引每 intervalRecords 条记录写一个点，首条总有点。索引可以丢弃并由日志重建。普通记录保持原布局。stamped record 在 timestamp 后、keyLength 前插入以下 64-byte producer identity: `marker:int32=-2|producerId:int64|producerEpoch:int32|firstSequence:int64|batchSize:int32|batchIndex:int32|batchHash:32bytes`。完整布局为 `length|crc32c|offset|timestamp|marker|producerId|producerEpoch|firstSequence|batchSize|batchIndex|batchHash|keyLength|valueLength|key|value`。最小总长为 96 bytes，1 MiB 上限包含 stamp。CRC32C 覆盖 offset 到最后一个 value byte，包含 stamp。普通 record 字节不变。已提供的 `ProducerStamp.ENCODED_OVERHEAD` 为 64 bytes，磁盘与 MessageCodec wire record 的 1 MiB 上限均包含这部分开销。

目录约定为 `<root>/<topic>/<partition>/<baseOffset:020d>.log`，配对索引后缀为 `.index`。保留范围是 `[logStartOffset,LEO)`，fetch 左闭右开；LEO 是下一条可写 offset，HW 是普通 consumer 能观察的最高提交边界（仅 `offset < HW` 可见）。group committed offset 也是 next offset，但不等于 HW。

A.1.2 TCP frame 与请求体

教学 frame 为 `length:int32|version:int16(1)|api:int16|correlationId:int32|error:int16|payload`，length 不含自身，范围 `[10,8,388,608]`。读到新 frame 首字节前 EOF 返回 null；frame 内 EOF 报 `INVALID_REQUEST`。请求 error 字段必须为 0。未知 version/API、过长 frame 和尾随 body 字节均拒绝。每条磁盘记录上限 1 MiB，不因 frame 上限而改变。

API id 固定如下：

ID	API	ID	API
1	METADATA	6	JOIN_GROUP
2	PRODUCE	7	HEARTBEAT
3	FETCH	8	LEAVE_GROUP
4	COMMIT_OFFSET	9	GROUP_ASSIGNMENT
5	FETCH_OFFSET	10	REPLICA_FETCH
11	IDEMPOTENT_PRODUCE		

MessageCodec 使用大端基础字段；字符串为 int32 UTF-8 byte 长度后接 bytes，集合为 int32 数量后逐项，Optional 值用 boolean 再接值，byte[] 用 int32 长度（仅 key 可 -1）。请求/成功响应字段按 provided Messages nested record 声明顺序编码；map 结果按 key 排序。Reply 头部 error=NONE 时 body 是相应成功 record；错误时 body 为 ErrorBody(message)，错误码只在 frame header 出现。字段 schema 由 `provided/.../protocol/Messages.java` 固定，学生不需重新定义十一种消息格式。FETCH 或 REPLICA_FETCH wire 上的 stamped RecordData 为 `marker=-2|producerId|producerEpoch|firstSequence|batchSize|batchIndex|batchHash[32]|keyLength|key|valueLength|value|timestamp`；普通 record 的 wire 编码不变。

A.1.3 消息 body 字段顺序

下面的请求与响应由 MessageCodec 按表内参数声明顺序编码。集合长度和字符串/byte 长度均为大端 int32；列表项无额外 tag，按各 value type 声明顺序内联。GroupToken nullable，用 boolean present 后跟 group、member、generation；Acks 为 int16，LEADER=1、ALL=-1。

API	请求记录与字段	成功响应记录与字段
METADATA	MetadataRequest: topic	MetadataBody: partitions
PRODUCE	ProduceRequest: tp, epoch, acks, timeoutMillis, records	ProduceBody: result
FETCH	FetchRequest: tp, epoch, offset, maxRecords, maxBytes, token	FetchBody: records, logStartOffset, logEndOffset, highWatermark, epoch

API	请求记录与字段	成功响应记录与字段
IDEMPOTENT_PRODUCE	IdempotentProduceRequest: tp, epoch, acks, timeoutMillis, producerId, producerEpoch, firstSequence, records	ProduceBody: result
COMMIT_OFFSET	CommitOffsetRequest: key, nextOffset, token	EmptyBody
FETCH_OFFSET	FetchOffsetRequest: key	OffsetBody: nextOffset (OptionalLong)
JOIN_GROUP	JoinGroupRequest: group, member, topic	GroupBody: assignment
HEARTBEAT	HeartbeatRequest: token	EmptyBody
LEAVE_GROUP	LeaveGroupRequest: group, member	GroupBody: assignment
GROUP_ASSIGNMENT	GroupAssignmentRequest: group, member	GroupBody: assignment
REPLICA_FETCH	ReplicaFetchRequest: tp, brokerId, epoch, fetchOffset, maxRecords, maxBytes, recoveryRead	ReplicaFetchBody: records, epoch, highWatermark, leaderLogEndOffset

Record	Fields in declaration order
TopicPartition	topic, partition
RecordData	key, value, timestamp, nullable producerStamp
LogRecord	offset, data
AppendResult	firstOffset, nextOffset
OffsetKey	group, tp
Endpoint	host, port
GroupAssignment	generation, assignments
PartitionMetadata	tp, leaderId, epoch, replicas, isr, leader
ProducerStamp	producerId, producerEpoch, firstSequence, batchSize, batchIndex, batchHash
BusinessEvent	eventId, account, delta

Each byte array is length-prefixed; only key may be null (length -1), while value is always present but may be empty.

OffsetStore entries use `MessageCodec.encodeOffsetEntry`, with null key and timestamp 0. The record's physical log offset is independent of committed nextOffset. BusinessEvent value 使用 `MessageCodec.encodeBusinessEvent` 编码: `version:uint8=1|eventId:identifier|account:identifier|delta:int64`。两个标识均为非空、严格 UTF-8 且最长 255 bytes; delta 可正、可负或为零。

A.1.4 标识符与值类型

topic、group、member 是非空 UTF-8 标识且最长 255 bytes；topic 只能包含 ASCII 字母数字、点、下划线、连字符，并拒绝 . 与 ... TopicPartition 的 partition 非负。RecordData 的 key 可以 null、value 不可 null（允许空数组），timestamp 非负；LogRecord offset 非负。数组所有权移交值对象后由接收方持有，不需要反复复制。所有 offset 表示下一条时都使用 half-open 边界，具体区别见正文各章。

A.2 幂等追加与持久化副作用

API id 11 是独立的 IDEMPOTENT_PRODUCE 请求；普通 PRODUCE 仍为非幂等模式，并拒绝带 stamp 的 client 输入。请求携带 tp, epoch, acks, timeoutMillis, producerId, producerEpoch, firstSequence, records；broker 为每条记录加入请求的 batch identity，并对记录数量与 timestamp/key/value 内容流式计算 SHA-256 指纹。producer id 与 epoch 由调用方管理。新 producer epoch 从 sequence 0 开始，且只有记录持久化后才生效。完全相同的已完成请求返回原 offset 区间，不追加；相同 identity 携带不同内容会冲突；sequence gap 与未知的旧 sequence 会被拒绝。匹配的部分 batch 可只追加缺失后缀来完成。派生状态从保留日志历史恢复；当 logStartOffset 非零时，幂等请求报 PRODUCER_STATE_EXPIRED。本课程没有 producer-state snapshot 或 client session WAL。

IdempotentProducer 每次调用只发送一次。调用失败会保留 pending batch；只有调用方显式调用 retry 才刷新 metadata，并以同一 identity、sequence 和 payload 重发一次。重复请求仍通过普通 ACK precheck 并等待确认，包括 ALL 的 ISR/HW/deadline 条件。这不改变从不自动重试的 SimpleProducer。

DurableEffectStore 将一条 BusinessEvent 作为一条本地日志记录 force 写入。首次 event id 会追加并更新账户余额；id 与内容都相同的重放返回 false，不再追加；同 id 不同内容或 checked balance 溢出会被拒绝。恢复从保留日志重建事件身份和余额，并要求日志起点为零。该本地账本不与 ProcessingLoop 的 offset commit 原子协调：commit 失败后 callback 可再次运行，而稳定 event id 可防止本地记账重复。group fencing 不能撤销已经发生的副作用。

A.3 固定错误码

Wire id 名称	用途
0 NONE	成功。
1 INVALID_REQUEST	参数、长度、版本或请求格式错误。
2 UNKNOWN_TOPIC_OR_PARTITION	topic/partition 不存在。
3 OFFSET_OUT_OF_RANGE	offset 超过当前日志起点/终点约束。
4 CORRUPT_RECORD	完整记录 CRC、格式或已提交历史冲突。
5 NOT_LEADER	请求者不是当前 leader。

Wire id 名称	用途
6 FENCED_EPOCH	请求使用旧 leader epoch。
7 NOT_ENOUGH_REPLICAS	当前 ISR 少于 minISR。
8 REQUEST_TIMEOUT	请求 deadline 到期，结果可能未知。
9 ILLEGAL_GENERATION	消费组 generation 已过期。
10 UNKNOWN_MEMBER	消费组 member 未登记。
11 NOT_ASSIGNED	操作涉及未分配 partition。
12 NO_ELIGIBLE_LEADER	无满足干净选举条件的候选。
13 STORAGE_ERROR	I/O 失败；保留底层 cause。
14 FENCED_PRODUCER_EPOCH	请求使用旧 producer epoch。
15 OUT_OF_ORDER_SEQUENCE	batch sequence 或边界无效。
16 DUPLICATE_SEQUENCE_C ONFLICT	重复 batch identity 的内容不同。
17 PRODUCER_STATE_EXPIR ED	所需 producer 历史已不在保留日志前缀中。
18 INCOMPLETE_BATCH	无关追加会跨越未完成的 producer batch。

未知 ID 不映射为成功；I/O 错误不吞 cause；网络错误 body 不回传服务器堆栈。

A.4 三十一歩与编辑入口

所有路径均相对于仓库根目录，学生修改 `exercises/src/main/java/io/simplekafka/` 下指定方法。每步累积命令为 `./gradlew :test -Pstep=N`；单步定位命令为 `./gradlew :stepTest -Pstep=N`，它不是晋级证明。

步 课程标题	编辑方法
1 记录编解码	<code>storage/RecordCodec</code> : encode, decode
2 日志追加	<code>storage/SegmentLog</code> : append
3 按位点读取	<code>storage/SegmentLog</code> : readFrom
4 崩溃恢复	<code>storage/SegmentLog</code> : recover
5 稀疏索引	<code>storage/SparseIndex</code> : add, floor, rebuild
6 分段日志	<code>storage/PartitionLog</code> : append, read
7 日志保留	<code>storage/PartitionLog</code> : deleteBefore
8 尾部截断	<code>storage/PartitionLog</code> : truncateTo
9 分区路由	<code>client/Partitioner</code> : choose
10 主题元数据	<code>broker/PartitionCatalog</code> : createTopic, metadata
11 TCP 帧协议	<code>protocol/FrameCodec</code> : write, read
12 单机 broker	<code>broker/BrokerHandler</code> : handle
13 批量生产者	<code>client/SimpleProducer</code> : send, flush

步 课程标题	编辑方法
14 拉取消费者	client/SimpleConsumer: assign, seek, poll
15 消费位点持久化	group/OffsetStore: commit, fetch; client/SimpleConsumer: resume, commitSync; broker/BrokerHandler: COMMIT_OFFSET, FETCH_OFFSET
16 至少一次处理	client/ProcessingLoop: runOnce
17 分区分配	group/RoundRobinAssignor: assign
18 成员与重平衡	group/GroupCoordinator: join, leave, assignment; broker/BrokerHandler: JOIN_GROUP, LEAVE_GROUP; GROUP_ASSIGNMENT
19 心跳与过期	group/GroupCoordinator: heartbeat, expire; broker/BrokerHandler: HEARTBEAT
20 消费组围栏	group/GroupCoordinator: validate; client/GroupConsumer: subscribe, poll, commitSync; broker/BrokerHandler: FETCH/COMMIT_OFFSET token fencing
21 拉取复制	broker/BrokerHandler: handle(REPLICA_FETCH); replication/FollowerReplicator: pollOnce
22 ISR 与高水位	replication/ReplicationTracker: report, expireLagging, advanceHighWatermark
23 生产确认	replication/AckPolicy: validateBeforeAppend, await
24 并发副本 broker	replication/ReplicatedPartition: produce, fetch
25 干净选举与 epoch	replication/LeaderElection: elect, checkLeader
26 分叉日志修复	replication/ReplicaReconciler: reconcile
27 元数据路由刷新	client/MetadataRouter: refresh, call
28 副本重新入 ISR	replication/ReplicaAdmission: tryAdd
29 三层一致性故障实验	lab/ConsistencyExperiments: producerOutcomeUnknown, leader-AckLoss, allAckTimeout, consumerReplay, staleGroupSideEffect
30 幂等生产者	client/IdempotentProducer: send, retry; replication/IdempotentAppender: append, recover; replication/ReplicatedPartition: produceIdempotent; broker/BrokerHandler: handle(IDEMPOTENT_PRODUCE); client/MetadataRouter: call; storage/RecordCodec: encode, decode; storage/SegmentLog: append, recover; storage/PartitionLog: append, deleteBefore, truncateTo
31 消费副作用幂等	client/DurableEffectStore: apply, recover

前七章各含四步；第 8 章含第 29–31 步。章节末累计通过点依次为 4、8、12、16、20、24、28、31。若前一步未通过，后一步测试通常也会失败，因为课程采用同一持续演进的系统而非互

不相干的演示程序。

A.5 课程简化与 Kafka 的差异

1. **路由**: key 使用 CRC32C unsigned modulo; Kafka 默认 key partition 算法不同。null key 采用实例级 round-robin, 而非 Kafka 的 sticky batching。
2. **磁盘**: 记录逐条编码并每 batch 显式 `force(false)`, 不是 Kafka record batch 与默认 page-cache 路径; 强制写入不保证媒介级断电安全。
3. **索引**: 每固定记录数写一个稀疏点, 不按 bytes 间隔。
4. **消费组**: 仅单 topic、即时重分配的简化模型, 不是完整 classic barrier、cooperative 重平衡或 KIP-848。token 围栏仅保护课程 broker 内 offset 提交, 不原子化外部副作用。
5. **offset**: 本地 OffsetStore 不是高可用复制压缩的 `__consumer_offsets`。
6. **复制与控制面**: 各 broker 用真实 TCP 拉取并写独立目录; authority/coordinator 却在同 JVM 且单一可信, 不是 KRaft, 不保证 controller/authority 故障后的集群恢复或任意分区下共识。
7. **可见性假设**: 本课程保留已提交前缀, 要求干净候选 LEO 不低于 authority HW, 使受控切换的已知提交边界不回退; 这比对真实 Kafka 跨 epoch HW 行为更强, 不可推广成 Kafka 永远单调的结论。
8. **重试、幂等与事务**: 普通 SimpleProducer 从不自动重试, timeout 可能已追加。独立的 IdempotentProducer 支持由调用方管理 producer identity 并显式重试, batch stamp 持久化在日志中; 但它不实现 Kafka PID 分配、transaction coordinator、client session WAL 或 producer-state snapshot, retention 可能使所需前缀过期。本地 DurableEffectStore 按稳定业务 event id 去重, 但与 consumer offset commit 不原子协调。没有 Kafka 事务或 `read_committed` 保证、compaction、fire-and-forget 或 ELR。at-least-once 容许重复; 本教学子集不声称端到端 exactly-once。
9. **恢复**: 第 26 步全量比较前缀为 $O(n)$ 教学简化; 真实系统利用 leader epoch 历史缩小分叉搜索范围。

A.6 官方资料与延伸阅读

以下资料用于解释 Kafka 概念与真实系统差异, 并非课程自定义 wire/storage 布局的规范。版本标签反映本书所引用页面; 不同版本配置和协议细节可能改变, 阅读时以所列页面自身版本为准。

主题	官方资料	关联章节
Kafka 设计与持久化模型	Kafka 4.2 Design	日志、复制、消费者位置的官方概览; 本课程简化项见上节。
消费位置与提交	KafkaConsumer API	第 14–16 章: position、seek、commit 与线程约束。

主题	官方资料	关联章节
最小 ISR 配置	Kafka 4.3 Topic Configs	第 22–24 章: <code>min.insync.replicas</code> 与写入确认。
消费组协议差异	Kafka 4.3 Rebalance Protocol	第 17–20 章: <code>classic</code> 与新协议背景; 课程不实现完整协议。
leader epoch 与截断	KIP-101	第 26 章: 为何仅靠 HW 不足以定位分叉。
JUnit 测试运行	JUnit 5.11.4 User Guide	阅读实际 Step 行为失败与测试报告。
Tectonic 编译选项	Tectonic compile	本书采用 XeTeX/CJK 与本地 cache 的构建入口。

A.7 排版与构建说明

默认主文件为英文版 `simpleKafka.tex`, 使用 `book` 文档类; 中文版 `simpleKafka-zh.tex` 使用 `ctexbook[UTF8,fontset=none,openany]`。共享 `preamble` 通过 `fontspec` 与 `xeCJK` 显式使用 TeX bundle 内的 `Latin Modern` 与 `Fandol` 字体文件, 不探测系统字体; `Noto`、`fontconfig`、`xelatex` 与 `latexmk` 都不是构建依赖。代码片段使用 `listings`, 不依赖 `minted`、`shell-escape` 或联网网页嵌入。`TikZ` 图由 TeX 本地绘制。EPUB 使用 `Pandoc 3.12.1` 自定义 `reader` 处理同一套展开后的 TeX 源码; `Tectonic` 渲染六张 `TikZ` 图, `Poppler` 的 `pdftoppm` 将其栅格化并嵌入 EPUB, 不维护第二份图形源码。

`./gradlew :setup` 解析经过 `checksum` 校验的 `JUnit`; `./gradlew :setupBook` 准备 `checksum` 校验的 `Tectonic 0.17.0` 与 `Pandoc 3.12.1`, 并检查 `Poppler` 的 `pdftoppm`。`Poppler` 需单独安装; `:setupBook` 不编译 Java 或解析 `JUnit`, 但 `Wrapper` 启动它仍需 Java 17+。`./gradlew :book` 默认生成英文版; `COURSE_LANG=zh ./gradlew :book` 生成中文版。`COURSE_LANG` 只接受 `en` 或 `zh`。英文 PDF/EPUB 位于 `build/book/en/simpleKafka.pdf` 与 `build/book/en/simpleKafka.epub`, 中文位于 `build/book/zh/simpleKafka-zh.pdf` 与 `build/book/zh/simpleKafka-zh.epub`。每次构建只更新所选语言在 `docs/book/dist/` 下的成品; 仓库克隆与源码 ZIP 已包含这四份文件, 读者无需 Java 或编译工具。首次构建在线获取 TeX bundle 与字体, `cache` 位于仓库 `.tools/tectonic-cache`; 两种语言都在线成功后, 可用 `BOOK_OFFLINE=1` 为 PDF 和图形构建启用 `Tectonic only-cached` 模式, `Gradle --offline` 则独立控制 `Gradle` 依赖解析。教材构建不编译 Java, 也不使用 `JUnit`。显式 `TECTONIC` 可提供兼容引擎; `PANDOC` 若设置必须为 `3.12.1`; `TEX_BUNDLE` 接受 URL、本地目录、`.zip` 或 `.ttb`, 不接受普通 `.tar`。首次 clone 离线构建不受保证; `bundle URL` 也不是完整资源集的内容摘要, PDF/EPUB 不承诺字节级可重复。

术语索引

术语	本书中的含义与位置
追加 (append)	在日志尾部按连续 offset 写入记录；第 1、2、6 章。
位点 (offset)	分区日志中的逻辑记录编号，不是文件字节位置；第 1 章。
字节位置 (position)	记录在一个 segment 文件内的物理起始位置；第 1、2 章。
LEO	Log End Offset，下一条可写记录的 offset；第 1、6 章。
HW	High Watermark，普通消费者可见的提交边界；第 6、7 章。
ISR	In-Sync Replicas，与当前 leader 保持在课程界定同步条件内的副本集合；第 6、7 章。
generation	消费组成员分配的版本号，用于拒绝旧成员提交；第 5 章。
leader epoch	分区 leader 角色版本；切换后旧角色请求被围栏；第 7 章。
至少一次	处理可能重放，但在提交之前不会悄悄越过未成功处理的记录；第 4 章。